Universität Siegen

Classification of Psychiatric Emergencies in Rescue Patients Using Hybrid Machine Learning and Transformer Models

Studienarbeit

Mechatronics

Somesh Salunkhe

Supervising Tutor:
Prof. Dr.-Ing. Habil Roman Obermaisser — Universität Siegen
M.Sc. Shad Ahammed — Universität Siegen

Declaration of Originality

I hereby declare that this project is my own original work and only the indicated sources and resources were used. All references, ideas and direct quotes taken from other sources and used in my project have been fully and properly cited.

Furthermore, it does not contain any material previously published or submitted, neither in whole nor in part, for credit of any other degree at any institution.

.....	.....
Ort, Datum / place, date	Unterschrift / signature

Abstract

The classification of psychiatric emergencies in rescue patients is a critical challenge in emergency medical services. Rapid and accurate diagnosis is essential to ensure timely intervention and appropriate medical care. This study presents a hybrid approach integrating machine learning (ML) and transformer-based natural language processing (NLP) models to enhance psychiatric emergency classification. The research leverages a dataset sourced from the KMU-Innovativ research project "KI-RETT," which includes over 273,000 rescue cases recorded between 2012 and 2021. Data preprocessing techniques, including cleaning, text parsing, and feature selection, were employed to prepare structured and unstructured medical records for analysis. Two classification models were implemented: a Random Forest Classifier and a Bidirectional Encoder Representations from Transformers (BERT) model. Feature engineering techniques such as word frequency analysis and vectorization (TF-IDF, BoW) were applied to improve predictive performance. The models were evaluated using metrics including accuracy, precision, recall, and F1-score, demonstrating the superiority of transformer-based NLP in emergency psychiatric classification. The trained models were deployed in an AI-powered assistant designed to assist emergency personnel with real-time decision support. This study highlights the potential of AI-driven classification systems in improving response times, reducing diagnostic errors, and enhancing emergency psychiatric care.

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
BERT	Bidirectional Encoder Representations from Transformers
DL	Deep Learning
NaN	Not a Number
EDA	Exploratory Data Analysis
TF-IDF	Term Frequency-Inverse Document Frequency
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
POS	Part-of-Speech
NER	Named Entity Recognition
RFECV	Recursive Feature Elimination and Cross Validation
RFE	Recursive Feature Elimination
SVM	Support Vector Machine
BoW	Bag-of-Words
NER	Named Entity Recognition

Table of Contents

1 Introduction **1**
1.1 Challenges in Emergency Medical Diagnostics . . . 1
1.2 Potential of AI in Emergency Medical Diagnostics . . . 2
1.3 Constraints and limitations of AI in Emergency Medical Diagnostics . 3
1.4 The Case for AI Adoption in Emergency Care . . . 4

2 Objective **5**
2.1 Methodology . . . 6
2.1.1 Data Preprocessing . . . 6
2.1.2 Feature Engineering . . . 6
2.1.3 Model Selection and Training . . . 7
2.1.4 Performance Evaluation . . . 7
2.1.5 Development of AI Assistant . . . 7

3 Literature Study **8**

4 Data Preprocessing **11**
4.1 KIRETT Dataset . . . 11
4.2 Data Segregation . . . 12
4.2.1 Psychiatric Data . . . 12
4.2.2 Non-psychiatric Data . . . 13
4.3 Data Cleaning . . . 15
4.3.1 Statistical Overview . . . 15
4.3.2 Dimensionality Reduction . . . 15
4.3.3 Data Formatting . . . 16
4.3.4 Handling Missing Values . . . 17
4.4 Merging Data Frames . . . 18
4.5 Text Parsing . . . 18
4.5.1 Natural Language Toolkit (NLTK) Library . . . 19
4.5.2 SpaCy . . . 19

5 Feature Engineering **21**
5.1 Feature Selection Techniques . . . 22
5.1.1 Machine Learning-Based Automated Feature Selection . . . 22
5.1.2 Custom Feature Engineering . . . 22
5.2 Word Frequency Analysis . . . 23
5.2.1 N-grams . . . 23
5.2.2 Unigrams . . . 24
5.2.3 Bigrams . . . 25

5.2.4 Trigrams . . . 25
5.2.5 Implementation . . . 25
5.2.6 Key Insights . . . 26
5.3 Comparative Word Frequency Distribution . . . 29
5.4 Contextual Text Analysis . . . 30
5.4.1 Bigrams as Features . . . 30
5.4.2 Composite Features . . . 31
5.5 Vocabulary Synonym Mapping . . . 32
5.6 Recursive Feature Elimination and Cross Validation (RFECV) . . . 32
5.7 Training Dataset . . . 33
5.7.1 Mappings . . . 33
5.7.2 Text and Target Separation . . . 34

6 Model Selection and Training 35
6.1 Random Forest Classifier . . . 36
6.2 Bidirectional Encoder Representations from Transformers (BERT) . . . 37
6.3 Model Training . . . 38
6.3.1 Random Forest Classifier . . . 39
6.3.2 BERT . . . 41

7 Performance Evaluation 45
7.1 Performance Evaluation: Random Forest Classifier . . . 45
7.2 Performance Evaluation: BERT . . . 46
7.3 Analysis of BERT's Improved Performance Over Random Forest . . . 50
7.3.1 Limitations of Random Forest Classifier in Text Classification 50
7.3.2 Advantages of BERT in Text Classification . . . 51

8 Development of AI Assistant 53
8.1 Assistant with Random Forest Classifier . . . 53
8.1.1 Pipeline . . . 53
8.2 Assistant with BERT . . . 55
8.2.1 Pipeline . . . 55

9 Conclusion 57

List of Figures I

Bibliography II

1 Introduction

Emergency medical situations are inherently challenging and often demand swift, accurate decision-making under pressure. Rescue personnel are tasked with diagnosing patients in a variety of unpredictable and high-stress scenarios, ranging from medical emergencies at home to accidents, assaults, and natural disasters. The nature of these emergencies often requires personnel to make rapid decisions to stabilize patients before transporting them to the hospital. This dynamic and demanding environment places considerable strain on the decision-making capabilities of rescue teams, exposing them to a range of operational and diagnostic challenges.

1.1 Challenges in Emergency Medical Diagnostics

The ability to perform accurate and timely diagnoses in emergency scenarios is hindered by a multitude of factors, including:

- **Time constraints:** The critical nature of emergencies allows little time for rescue personnel to perform thorough patient assessments or deliberate on treatment plans. Decisions must often be made within minutes, increasing the risk of diagnostic errors and delayed interventions.[2][11]
- **Limited diagnostic tools:** Rescue operations are typically carried out in resource-constrained settings, where advanced diagnostic equipment is unavailable. Instead, personnel must rely on clinical presentations, basic vital signs, and their own expertise to make potentially life-saving decisions.[1]
- **Incomplete patient information:** Rescue personnel frequently operate with minimal background on the patient's medical history, pre-existing conditions, or current medications, which can hinder the diagnostic process and increase uncertainty.[11]
- **Environmental challenges:** Emergency scenarios often occur in difficult environments—such as disaster zones, accident sites, or remote areas—where external factors may interfere with accurate patient evaluation and care delivery.[11]
- **Risk-benefit analysis:** Personnel must balance the urgency of diagnostic procedures against resource availability and potential risks to both the patient and the rescue team.[20]
- **Fatigue and stress:** The physically and mentally taxing nature of emergency rescue work, often involving prolonged shifts and high-stakes situations, can lead to cognitive fatigue and impaired decision-making.[2]

- **Varying disease presentations:** Many diseases and medical conditions present with diverse symptoms that can be easily misinterpreted without the aid of comprehensive diagnostic tools or specialized expertise.
- **Non-specific symptoms:** Many medical emergencies, such as infections, cardiovascular issues, or trauma, often manifest with vague or non-specific symptoms, making accurate diagnosis particularly challenging.[17]
- **Special patient populations:** Diagnosing specific patient groups, such as older adults, children, or individuals with comorbidities, presents additional challenges due to atypical clinical presentations, age-related factors, and polypharmacy effects.[8]

These challenges highlight the pressing need for innovative solutions to improve the diagnostic performance and decision-making capabilities of rescue personnel in emergency settings.

1.2 Potential of AI in Emergency Medical Diagnostics

Artificial intelligence (AI) has emerged as a promising solution to address the diagnostic challenges faced by rescue personnel. AI algorithms, powered by machine learning and deep learning technologies, have demonstrated their capability to analyze vast amounts of medical data and provide accurate predictions in a fraction of the time required by traditional methods. The integration of AI into emergency medical diagnostics has the potential to revolutionize the field by enabling faster, more reliable decision-making. Some of the key advantages of adopting AI in emergency rescue operations include:

- **Enhanced decision support:** AI systems, trained on extensive datasets encompassing a wide range of medical conditions, can provide rapid and reliable diagnostic suggestions. These systems assist rescue personnel in making informed decisions, even in high-pressure scenarios.
- **Improved diagnostic accuracy:** Advanced machine learning models, such as Extreme Gradient Boosting (XGB), have achieved remarkable success in predicting complex medical conditions like cardiovascular and respiratory complications, with accuracy levels exceeding 90%.[6]
- **Integration of diverse data sources:** AI can analyze heterogeneous data types, including electronic health records, vital signs, laboratory results, and even emerging diagnostic tools like chemical vapor sensors (eNoses). This ability enhances diagnostic precision and reduces uncertainty.[6]
- **Reduction of diagnostic errors:** AI systems have demonstrated the potential to significantly reduce diagnostic errors, including missed or delayed diagnoses, which are prevalent in high-stress healthcare settings.[6]
- **Support for non-specialist personnel:** Rescue personnel often lack specialized medical training. AI tools can bridge this gap by providing expert-level

diagnostic insights, empowering personnel to make accurate assessments even in the absence of advanced medical expertise.

- **Fatigue mitigation:** Unlike human responders, AI systems maintain consistent performance and accuracy regardless of workload, stress, or external environmental factors.[16]
- **Rapid data processing:** AI algorithms excel at processing and interpreting medical data, such as imaging and laboratory results, much faster than traditional diagnostic methods. This speed is critical in emergencies, where time is of the essence.
- **Reduced cognitive overload:** By automating routine diagnostic tasks, AI frees up mental resources for rescue personnel, allowing them to focus on critical decisions and patient-specific care.
- **Continuous learning and adaptability:** AI systems can be updated regularly with new medical knowledge, enabling them to stay aligned with the latest diagnostic and treatment protocols.

1.3 Constraints and limitations of AI in Emergency Medical Diagnostics

AI's integration into emergency medical diagnostics faces significant constraints that limit its current effectiveness and reliability. While the technology shows promise for enhancing decision-making in time-sensitive scenarios, several critical challenges must be addressed before widespread clinical adoption becomes feasible.

1. **Diagnostic Accuracy Limitations:** Current AI systems struggle to match human diagnostic performance in complex emergency cases. Despite processing vast datasets, these tools:
   - Fail to account for nuanced clinical contexts that experienced physicians intuitively consider.[14]
   - Demonstrate reduced accuracy when confronted with atypical symptom presentations or rare conditions.[14]
   - Lack capacity for holistic patient evaluation beyond algorithmic pattern recognition.[13]
2. **Ethical and Legal Constraints:** Accountability gaps create substantial implementation barriers:
   - No clear legal framework determines liability for AI diagnostic errors (developer vs clinician).[13][14]
   - Current malpractice insurance models don't adequately cover AI-assisted decisions.[14][9]
   - Informed consent complexities arise regarding AI's role in life-threatening situations.[14][9]

3. **Algorithmic Bias and Equity Concerns:** Training data limitations create systemic risks:
   - Under-representation of minority groups in datasets leads to diagnostic disparities.[14][9]
   - Spectrum bias causes performance degradation in real-world populations vs controlled trials.[14]
   - Institutional data collection practices may perpetuate existing healthcare inequities.[14][9]
4. **Infrastructure and Implementation Barriers:** Practical deployment challenges include:
   - Integration difficulties with legacy hospital IT systems.[3]
   - High computational resource requirements for real-time analysis.[3]
   - Staff training needs and workflow disruption concerns.[9][3]

While AI continues to evolve as a diagnostic adjunct, emergency medicine's high-stakes environment demands solutions that address these limitations through improved technical reliability, ethical governance frameworks, and enhanced human-AI collaboration models. The technology currently serves best in supporting documentation and triage prioritization rather than autonomous decision-making.[3]

1.4 The Case for AI Adoption in Emergency Care

The integration of AI into emergency rescue operations represents a significant step forward in addressing the limitations faced by rescue personnel. AI systems are uniquely positioned to complement human expertise, offering support that is both rapid and reliable. By leveraging AI, rescue teams can achieve earlier diagnoses and initiate timely interventions, ultimately improving patient outcomes and reducing mortality rates. Furthermore, the ability of AI algorithms to process complex datasets with precision and robustness makes them an indispensable tool in emergency care. Their adoption has the potential to transform the field, ensuring that even in the most resource-constrained or high-pressure situations, rescue personnel are equipped with the tools they need to deliver effective care. As research and development in AI continue to advance, it is imperative to explore and implement AI-driven solutions in the medical domain. By harnessing the power of AI, we can not only enhance the efficiency and accuracy of emergency care but also alleviate the burden on rescue personnel, paving the way for a more resilient and responsive healthcare system.

2 Objective

This study explores the development of an AI assistant leveraging Machine Learning (ML) and Deep Learning (DL) techniques to support medical emergency rescue personnel in their decision-making process. The primary focus of the project is on cases involving psychiatric patients, a group that often presents unique diagnostic challenges in emergency scenarios. The goal is to design an AI-driven assistant capable of analyzing medical data and providing accurate, real-time classifications to aid rescue personnel. The assistant will be trained on a rescue emergency dataset, ensuring it is tailored to the specific demands of this critical domain. To enhance usability, the system is designed to interact with rescue personnel through natural language, bridging the gap between complex computational processes and practical, on-the-ground application. To achieve these objectives, two core models form the foundation of the AI assistant:

- **Random Forest Classifier:** This machine learning model serves as a robust, interpretable baseline for processing structured medical data. Its ability to handle feature importance and mitigate overfitting makes it an ideal choice for initial classification tasks.
- **BERT:** The Bidirectional Encoder Representations from Transformers (BERT) model is employed to handle natural language understanding and communication. BERT serves as the intermediary between the classification engine and the rescue personnel, enabling the assistant to interpret queries, generate responses, and facilitate seamless interaction in human language.

The system operates as a standalone AI assistant designed specifically for emergency personnel. Users input the patient's medical data into the system, and the assistant processes the information to classify the patient into one of two categories: Psychiatric or Non-psychiatric. This classification helps rescue personnel quickly identify cases where psychiatric symptoms are present, enabling faster and more focused intervention.

Key Features of the AI Assistant

- **Real-Time Decision Support:** The assistant delivers rapid classifications based on the entered medical data, ensuring personnel can make informed decisions under time constraints.
- **Natural Language Interface:** By integrating the BERT transformer, the assistant is equipped with the capability to converse with rescue personnel, simplifying the interaction and making the system accessible even for non-technical users.

- **Psychiatric Case Focus:** The assistant prioritizes cases involving psychological disorders, an area where early diagnosis and treatment are critical for patient outcomes.
- **Tailored Dataset Training:** The system is trained on a specialized medical emergency dataset to ensure high accuracy and relevance to real-world scenarios.

Impact on Emergency Care

The implementation of this AI assistant has the potential to significantly improve outcomes in emergency psychiatric cases. By automating the classification of psychiatric and non-psychiatric patients, the system reduces the cognitive load on rescue personnel, enabling them to act more quickly and efficiently. Early identification of psychiatric symptoms not only facilitates timely treatment but also reduces the risk of misdiagnosis or delayed care. In addition to improving response times, the natural language interface ensures ease of use, allowing the assistant to seamlessly integrate into the workflow of rescue operations. By providing an intuitive, accurate, and reliable diagnostic tool, the AI assistant empowers rescue personnel to deliver better care, even in high-pressure environments. This study underscores the transformative potential of AI in emergency healthcare, paving the way for innovative solutions that enhance both the efficiency and quality of medical services.

2.1 Methodology

The methodology section outlines the systematic approach adopted to develop an AI assistant capable of classifying psychiatric emergencies in rescue patients. The process involves several key stages, including data preprocessing, feature engineering, model selection, training, and evaluation. The following sections provide a detailed description of each step in the methodology.

2.1.1 Data Preprocessing

Data preprocessing is a critical step in ensuring the quality and usability of the dataset for machine learning models.The preprocessing steps include:

1. Data Segregation
2. Data Cleaning
3. Text Parsing

2.1.2 Feature Engineering

Feature engineering is the process of transforming raw data into features that can be used by machine learning models. The following steps were taken to engineer features from the preprocessed data:

1. Word Frequency Analysis
2. Composite Features
3. Synonym Mapping
4. Recursive Feature Elimination and Cross Validation (RFECV)

2.1.3 Model Selection and Training

Two models were chosen and trained for this study:

1. Random Forest Classifier : The Random Forest algorithm is an ensemble learning method that combines multiple decision trees to improve classification accuracy. The model was trained on the preprocessed and feature-engineered dataset. The text data was vectorized using the Bag-of-Words (BoW) approach, which converted the text into numerical representations suitable for the Random Forest algorithm.
2. BERT : BERT is a transformer-based model that excels in natural language understanding tasks. Unlike the Random Forest model, BERT does not require explicit feature engineering. Instead, it processes raw text and generates contextual word embeddings. The BERT model was fine-tuned on the dataset using the Hugging Face Transformers library. The text data was tokenized and converted into tensors, which were then used to train the model.

2.1.4 Performance Evaluation

The performance of both models was evaluated using standard metrics such as accuracy, precision, recall, and F1-score. The evaluation was conducted on a separate test dataset that was not used during training. The results were compared to determine which model performed better in classifying psychiatric emergencies.

2.1.5 Development of AI Assistant

The trained models were deployed into an AI-powered assistant designed for real-time classification of psychiatric emergencies. The assistant utilizes a natural language interface powered by BERT to facilitate interaction with rescue personnel, allowing them to input patient data and receive diagnostic classifications.

3 Literature Study

To deepen our understanding of the dataset from a medical perspective, we conducted a literature review. This step was essential for identifying which of the 452 unique field attributes could contribute to the diagnosis of patients, specifically in identifying mental health conditions such as depression, anxiety, and other psychiatric disorders. The insights from the literature study guided our selection of relevant features, which were crucial for developing a robust machine learning model. The findings from the literature are summarized as follows:

1. The paper by Welch and Carson (2018) provides a critical examination of the intersection between psychiatric symptoms and underlying medical or neurological conditions, emphasizing the diagnostic challenges in distinguishing primary psychiatric disorders from organic etiologies. The authors highlight that psychiatric diagnoses remain syndromal due to the lack of definitive biomarkers, increasing the risk of misattributing symptoms like agitation, psychosis, or emotional disturbances to psychiatric causes when they may stem from conditions such as delirium, autoimmune encephalitis (e.g., NMDA receptor antibody encephalitis), or neurodegenerative diseases (e.g., frontotemporal dementia). Delirium is underscored as a key consideration in acute medical settings, characterized by abrupt onset, fluctuating cognition, and attentional deficits, with hypoactive forms often overlooked or misdiagnosed as depression. The paper stresses the importance of systematic evaluation, including cognitive testing (e.g., 4AT screening), neurological exams, and basic laboratory workups (e.g., metabolic panels, drug screens), to identify red flags such as atypical age of onset, rapid symptom progression, or unexplained neurological signs. The authors also caution against premature closure in cases with psychiatric comorbidities, advocating for vigilance in reassessing patients with persistent or evolving symptoms. Practical guidance is provided on differentiating features (e.g., auditory hallucinations in schizophrenia vs. visual hallucinations in delirium) and integrating tools like the Addenbrooke's Cognitive Examination III to detect subtle cognitive impairments. The review underscores the clinical imperative to avoid diagnostic overshadowing, particularly given the high morbidity associated with missed organic diagnoses, and calls for collaboration between medical and psychiatric teams to optimize outcomes.

2. The article by McKee and Brahm (2016) addresses the challenge of identifying underlying medical conditions that manifest as psychiatric symptoms, a phenomenon termed "medical mimic" or "secondary psychosis." The authors highlight the difficulty in differentiating between primary and secondary psychosis, especially in patients with existing psychiatric conditions, emphasizing that no single test can definitively distinguish them. They propose a structured clinical framework for evaluating potential medical mimics, focusing on atypical features of psychiatric symp-

toms, risk factors, and common syndromic presentations. The review underscores the importance of a comprehensive evaluation, including medical and psychiatric history, physical examination, medication review, mental status examination, and laboratory testing, to avoid misdiagnosis and ensure appropriate treatment, ultimately improving patient outcomes and reducing healthcare costs.

3. This review by Jung et. al. (2019) explores the relationship between heart rate variability (HRV) and psychiatric illnesses, specifically major depressive disorder (MDD), schizophrenia, and posttraumatic stress disorder (PTSD), focusing on their shared symptom of cognitive dysfunction. HRV, a noninvasive measure of autonomic nervous system activity, reflects the dynamic interplay between the sympathetic and parasympathetic nervous systems, with decreased HRV often indicating autonomic dysfunction. The review examines how HRV is characterized in patients with MDD, schizophrenia, and PTSD compared to healthy controls, highlighting the connection between the heart and brain through HRV's interaction with quantitative electroencephalography (qEEG) during sleep cycles. Ultimately, it investigates the association between HRV and cognitive function, suggesting that reduced HRV is a common manifestation in these disorders and is closely linked to changes in qEEG and individual differences in cognitive performance, suggesting that studying the physiological and cognitive mechanisms related to HRV may improve our understanding of these conditions.

4. This issue of Alcohol Research: Current Reviews (ARCR) by Kwako et. al. focuses on the co-occurrence of alcohol use disorder (AUD) and various mental health conditions, investigating how this co-occurrence impacts symptom severity, prognosis, and treatment outcomes. It covers a range of co-occurring disorders, including depressive disorders, suicidal behavior, anxiety, personality disorders (antisocial and borderline), and schizophrenia spectrum disorders. The reviews discuss prevalence rates, potential explanations for comorbidity, neurobiological perspectives, and the challenges in diagnosis and treatment due to overlapping symptoms. Additionally, the issue explores the efficacy of different treatment approaches and interventions, emphasizing the importance of integrated treatment for both AUD and co-occurring mental health conditions.

5. This systematic review by Shahimi et al. investigates the association between mental illness and blood pressure variability (BPV). It examines the methods of BPV measurements and analysis. The review included twelve studies that measured short-term, long-term, and ultra-short-term BPV in individuals with mental illnesses such as anxiety, depression, panic disorder, and hostility, excluding those with hypertension. The review found that mental illness is significantly associated with increased BPV, particularly in younger and middle-aged adults, regardless of the BPV measurement duration. Early therapeutic intervention in mental illness may prevent autonomic dysregulation-associated diseases and reduce negative cardiac outcomes.

6. The article by Shivani et. al. addresses the complexities clinicians face when evaluating psychiatric complaints in patients with alcohol use disorders. It highlights that heavy drinking can coexist with, contribute to, or result from various psychiatric syndromes, complicating accurate diagnosis. The authors propose an algo-

rithm to distinguish between alcohol-related psychiatric symptoms, alcohol-induced syndromes, and independent psychiatric disorders commonly associated with alcoholism, emphasizing the importance of considering gender, family history, and the course of illness over time. They stress that clinicians should remain flexible with diagnoses and monitor abstinence to improve diagnostic accuracy and reduce illness severity in alcoholic patients, as comorbidity increases the risk of relapse, suicide attempts, and mental health service utilization.

7. This article by Trivedi reviews the significant connection between depression and physical symptoms, highlighting how these symptoms, such as joint pain, gastrointestinal issues, and fatigue, often present as the primary complaint in primary care settings. The author emphasizes that depression can be overlooked when patients present solely with somatic symptoms, leading to potential misdiagnosis and inadequate treatment. The article explores the neurological basis for this link, pointing to the dysregulation of neurotransmitters like serotonin and norepinephrine, which influence both mood and pain perception. Furthermore, it discusses the importance of incorporating physical symptoms into treatment goals, advocating for the use of dual-action antidepressants and thorough assessment tools to achieve full remission and prevent relapse in depressed patients.

4 Data Preprocessing

The quality of data is a fundamental determinant in the effectiveness and accuracy of machine learning models. It directly influences how well models can generalize and make reliable predictions. In the realm of data science and machine learning, data preprocessing is not merely a preliminary task, but an essential phase that sets the foundation for model performance. A meticulously preprocessed dataset enables the machine learning model to effectively capture patterns and relationships in the data, leading to accurate and reliable results. However, a poorly preprocessed dataset can introduce noise, inconsistencies, or irrelevant information, which can result in misleading predictions and erroneous conclusions. Consequently, ensuring that the data is properly preprocessed is a critical step in building robust and reliable machine learning systems.

4.1 KIRETT Dataset

The dataset utilized in this research was obtained through the KMU-Innovativ research project "KIRETT" in Germany, aimed at identifying rescue scenarios to accelerate the treatment pathways for patients. The dataset was generously provided by the Siegen-Wittgenstein rescue station, encompassing rescue operations carried out between January 2012 and August 2021 within the city of Siegen, Germany [2]. This dataset contains anonymized information from 273,183 unique rescue events, representing a wide spectrum of medical emergencies. Each rescue event is documented with more than 452 unique attributes, capturing critical aspects of rescue operations and patient information. The records include:

- Rescue protocols employed during the operations.
- Location of rescue events.
- Clinical records of patients, including health prognoses and vital signs.
- Initial impressions and assessments made by the rescue personnel.
- Treatments performed during the emergency.

The data offers a comprehensive view of historical rescue operations, capturing the challenges faced during emergency scenarios. However, given the nature of data collection in high-pressure, emergency environments, certain limitations in data quality were observed. For instance:

- Misspelled text entries: Errors due to manual input during rescue situations.

- Abnormal or inconsistent vital sign values: Data such as blood glucose, respiratory rate, and pulse rate occasionally displayed outliers or inaccuracies caused by rapid, high-stress data acquisition.

Despite these challenges, the dataset remains a rich source of information for analyzing rescue scenarios and developing AI-based solutions to support emergency personnel. Its large scale and variety of attributes make it a valuable resource for training and evaluating machine learning models aimed at improving emergency response and patient outcomes.

4.2 Data Segregation

The database obtained from the Siegen-Wittgenstein rescue station, as described earlier, was collected in emergency situations. Consequently, the dataset was completely unprocessed, containing raw information with several challenges such as spelling errors, missing data, and inconsistencies in patient vitals. The data was stored in two separate Excel files:

1. **MasterfileComplication.xlsx:** This file contains information about the complications observed in each patient, including diagnosed diseases, illness types, and injury details.
2. **Combined Data.xlsx:** This file contains health-related information recorded during both the rescue period and treatment phase, including patient vitals, test results, and clinical assessments.

Both tables share a common primary key, namely the 'case' column, which contains the unique case numbers for individual patients. This key was used to merge and query data across the two files to create a unified dataset for analysis. Since this study focuses on psychiatric disorders, the database was divided into two distinct subsets:

- Psychiatric Data: Cases where a psychiatric disorder was diagnosed.
- Non-Psychiatric Data: Cases where all other diagnoses apart from psychiatric disorders were recorded.

This segmentation allowed for a more focused and balanced dataset to train machine learning models effectively.

4.2.1 Psychiatric Data

The original dataset comprises approximately 273,183 patient cases, with diagnoses spanning several medical conditions such as abdominal diseases, cardiovascular diseases, psychiatric diseases, respiratory diseases etc. However, a distribution analysis revealed that the data was highly imbalanced, with only around 12,079 cases classified under psychiatric disorders, while the remaining belonged to non-psychiatric conditions. Using the entire dataset without addressing this imbalance would have

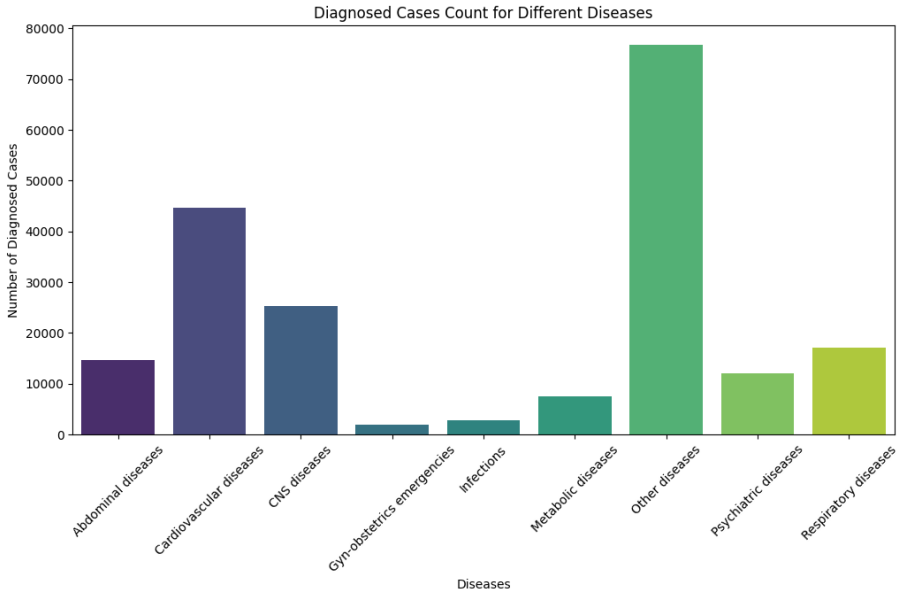

Figure 4.1: Diagnosed Cases Count for Different Diseases.

negatively impacted the classification task. For example, a machine learning model trained on such skewed data might achieve a superficial accuracy of over 96% simply by predicting all cases as non-psychiatric (negative class), resulting in poor predictive performance for psychiatric cases. To address this, the psychiatric data was separated using the following procedure:

1. **Data Import:** Both Excel files, MasterfileComplication.xlsx and Combined Data.xlsx, were imported into Python and stored as Pandas Data Frames.
2. **Filtering Psychiatric Cases:**
   - The "Psychiatric diseases" field in the complications table was used as the key attribute.
   - This field contains specific psychiatric diagnoses (e.g., "depression"). If a patient had a psychiatric diagnosis, the field contained the disease name; otherwise, it was empty (NaN).
   - Cases with non-empty values in the "Psychiatric diseases" field were retained, while others were discarded.
3. **Merging Tables:** The filtered psychiatric cases were merged with the corresponding data in the health information table using the common primary key ('case').
4. **Final Dataset:** The resulting Data Frame contained 12,079 cases of psychiatric disorders, which served as the positive class for further analysis.

4.2.2 Non-psychiatric Data

The dataset for non-psychiatric cases initially contained approximately 261,000 cases. Including all these cases in the final dataset would have skewed the class

balance, resulting in a disproportionate representation of the negative class. To maintain a balanced dataset with a 50-50% distribution between psychiatric and non-psychiatric labels, only 12,079 cases were selected from the non-psychiatric dataset. The selection process was as follows:

1. **Filtering Non-Psychiatric Cases:** Cases with empty or NaN values in the "Psychiatric diseases" field were retained, as these indicated the absence of psychiatric disorders.
2. **Merging Tables:** The filtered non-psychiatric cases were merged with the health information table using the 'case' column.
3. **Sorting by Data Completeness:**
   - The merged dataset was sorted based on the number of NaN values per case, from least to most.
   - This ensured that cases with the most complete and reliable data were prioritized for inclusion in the final dataset.
4. **Subsampling:** The top 12,079 cases from the sorted dataset were retained, while the remaining cases were discarded.

Although discarding data is generally discouraged in machine learning, this step was necessary to maintain balance between the two labels.

Summary

The final raw dataset was prepared with a balanced distribution of 12,079 cases for each class:

- Psychiatric cases: Positive label.
- Non-psychiatric cases: Negative label.

This balanced dataset ensures fair training and evaluation of the machine learning models, reducing the risk of biased predictions and improving the overall performance of the classification task.

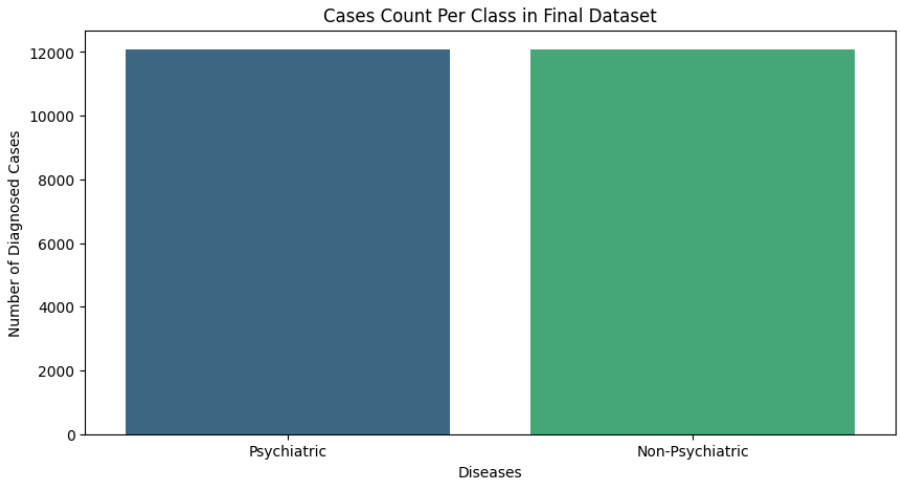

Figure 4.2: Cases Count Per Class in Final Dataset.

4.3 Data Cleaning

Data cleaning is an essential and comprehensive process that involves applying various techniques to transform raw, unstructured data into a structured, usable format for analysis. This step is vital for ensuring that the data is both accurate and consistent. The techniques used in data cleaning include handling missing values, removing duplicate entries, converting data types, addressing inconsistencies, and correcting errors in the dataset. Each technique is selected and applied based on the specific characteristics of the data and the requirements of the analysis.

Skipping the data cleaning stage can significantly hamper the performance of machine learning models. Many models cannot handle missing values effectively, and some algorithms are sensitive to noise, outliers, or high-dimensional data. Without proper preprocessing, such issues can lead to poor model performance and unreliable results.

Handling missing data, for example, may involve strategies like imputation or removal depending on the context and impact of the missing information. Removing duplicates ensures that the dataset doesn't contain redundant entries that could distort the analysis, while data type conversion ensures that numerical values are correctly identified and treated, making them suitable for machine learning algorithms.

Therefore, data cleaning is therefore indispensable to ensure that the dataset is clean, complete, and accurately formatted, which is critical for model training and making accurate predictions.

4.3.1 Statistical Overview

To gain an initial understanding of the dataset, we began by computing basic statistics. This included determining the total number of rows and columns in the data frame, as well as the mean, maximum, and minimum values for the numeric columns. In addition, we assessed the distribution of different data types present in the dataset, including both numeric and categorical (string) data. This initial exploration helped us understand the structure of the dataset and identified any immediate concerns related to missing or inconsistent values.

4.3.2 Dimensionality Reduction

The dataset initially contained 101 data columns, each containing diverse data, resulting in a high dimensionality that made it challenging to manage. Therefore, reducing the dimensionality of the dataset was necessary for efficient processing and to avoid issues such as overfitting or model complexity.

Through the exploration of the basic statistics of the dataset, it became evident that many fields contained NaN values or had sparse data for the majority of patient instances. After evaluating the proportion of missing data, it was determined that

any field containing missing values for more than 50% of the instances would not contribute meaningful information. As such, these fields were removed from the dataset.

To identify these columns, a function was implemented to calculate the number of missing (NaN) values for each column. This function leveraged the built-in methods of the pandas Data Frame object, specifically isna(), sum(), and len(). Using these methods, we were able to quickly identify columns with a significant proportion of missing values.

The analysis revealed that 77 of the 101 columns contained more than 50% missing data. However, discarding all these columns outright was not a feasible option, as some fields identified as critical in the literature study for psychiatric diagnosis also had missing data. Therefore, only the columns that were not deemed important for psychiatric diagnosis, based on the literature review, were discarded. This process reduced the number of fields from 101 to 31, representing a significant reduction in dimensionality.

Although this reduction in features was substantial, the dataset still had 31 fields, which remained too large for direct use in machine learning models. At this stage, further dimensionality reduction was not carried out, as it was still uncertain which fields would be most valuable for feature extraction. Therefore, the next phase of dimensionality reduction will occur during the feature extraction and feature engineering stages, where we will determine which fields contribute most to the predictive accuracy of the model.

4.3.3 Data Formatting

The first step in cleaning the data involved addressing the formatting issues. As previously mentioned, the data was stored in an inconsistent manner: everything was treated as text, regardless of whether it was date, time, numeric data, or plain text. Additionally, the format for text data varied: some fields contained plain text, while others had text enclosed in single quotes or square brackets.

To standardize the format, two Python functions were defined to clean the text in all relevant columns of the dataset. These functions removed any extraneous symbols, quotes, or punctuation at the beginning and end of the text. After cleaning the text, it was necessary to handle other data types that had been stored improperly.

For instance, date and time information was initially stored in a single field, which required separation into two distinct fields: one for the date and one for the time. The data in these fields was then converted into the correct format using the built-in pandas Data Frame method to_datetime().

Next, numeric data was converted to its appropriate data type. To achieve this, the pandas method to_numeric() was employed to ensure that all numeric data was properly recognized as numeric types.

Removing Unwanted White Spaces

In addition to removing punctuation and brackets, another significant issue was the presence of unwanted white spaces throughout the text due to inconsistent formatting during data entry. These extra white spaces could cause issues during feature engineering, so they needed to be removed. To handle this, the built-in Python method strip() was used to eliminate any leading or trailing spaces from the text, ensuring a cleaner and more standardized dataset for further processing.

4.3.4 Handling Missing Values

With the data now correctly formatted, the next critical step was to address the issue of missing values. As discussed earlier, the dataset contained many fields with missing or incomplete data. This was due to the nature of the data collection, where certain information was unavailable, some personnel lacked the necessary equipment, or time constraints prevented the complete recording of all fields. Since machine learning algorithms do not accept training dataset with missing data, it was essential to take steps to handle these gaps. The built-in pandas data Frame method fillna() was used to fill in the missing values.

However, filling missing data with random values could skew the dataset, especially for numeric fields. Similarly, filling text fields with random data would not be meaningful. Therefore, the approach to handling missing values required careful consideration and the application of appropriate methods for each data type.

Standard Techniques for Handling Missing Data

There are several standard techniques used in the industry to handle missing data, including:

1. Removing Instances Containing Missing Values
2. Imputation Methods:
    - Mean, Median, Mode Imputation
    - Fill Forward, Fill Backward
3. Interpolation Methods

The choice of method depends on the data type and the specific requirements of the dataset. Given that our dataset contains a mix of data types, different methods were employed for numeric and text data.

Missing Data Handling in Numeric Fields

Imputation methods are commonly used to handle missing numeric data, but selecting the right approach requires careful consideration of the impact on the results. Our dataset contained various types of numeric data, each with its own characteristics. For example, fields such as height and weight can usually be imputed using the mean value, as this approach doesn't distort the data significantly. However, for critical health indicators like heart rate and blood pressure—key to diagnosing psychiatric conditions—simple imputation methods like mean substitution may not

be appropriate. In these cases, different strategies were applied to handle missing values:

- Basic health data such as height and weight were imputed using the mean of the existing values.
- Sensitive data such as pain scale scores, blood pressure, and heart rate were imputed with a specific placeholder value (e.g., -1). This is because it is impossible for these values to be negative, so -1 served as a flag to indicate that the data was missing without distorting the dataset.

Missing Data Handling in Text Fields

Handling missing data in text fields was similarly important. Machine learning algorithms cannot process datasets with empty text fields and filling them with random data would not provide meaningful information. Therefore, missing text data was filled with the placeholder text "unknown." This ensured that missing values were marked appropriately and would not be mistaken for valid data during feature engineering.

It is crucial to note that methods such as fill forward or fill backward were avoided because they could introduce bias into the dataset. These methods fill empty cells with data either in the previous cell or in the next cell, this could distort the data and could lead to bias in the case of categorical data. This affects subsequent feature engineering.

4.4 Merging Data Frames

Once the psychiatric and non-psychiatric datasets had been preprocessed and cleaned, they needed to be merged to create the final dataset for feature engineering. However, direct merging of the two datasets posed a problem: there would be no way to distinguish between psychiatric and non-psychiatric cases after the merge. To resolve this, labels were added to the datasets to indicate the class of each case.

For machine learning purposes, these labels needed to be in numeric format, as algorithms do not process text labels. Thus, a numeric labeling scheme was implemented: psychiatric cases were assigned a label of 1 (indicating "positive"), and non-psychiatric cases were labeled with 0 (indicating "negative"). Once all instances were labeled accordingly, the two datasets were merged, resulting in the final dataset.

4.5 Text Parsing

Text parsing is a crucial aspect of natural language processing (NLP), which involves analyzing and transforming raw text data into structured information that can be understood by machines. In the context of AI development, effective text parsing is vital for extracting meaningful data, enabling the AI to interpret and respond to natural language input. This process adheres to the rules of formal grammar,

making it essential for the proper understanding and processing of both natural and computer languages.

The core components of text parsing are:

1. **Tokenization:** Splitting text into individual words or tokens, which can then be analyzed further.
2. **Syntactic Analysis:** Determining the grammatical structure of the text, which helps in understanding the relationship between words.
3. **Semantic Analysis:** Extracting the meaning of the text, which is critical for understanding context and intent.

4.5.1 Natural Language Toolkit (NLTK) Library

The Natural Language Toolkit (NLTK) [7] is a robust library in Python designed for NLP tasks. First introduced in 2001 by the University of Pennsylvania, NLTK provides a wide range of tools and resources for processing text and building complex language models. It offers comprehensive functionality for tasks such as tokenization, syntactic parsing, and semantic analysis.

The primary features of NLTK include [7]:

- **Tokenization:** Breaking down text into individual words or sentences, allowing for easier analysis.
- **Part-of-Speech (POS) Tagging:** Assigning grammatical categories (such as noun, verb, etc.) to words in the text.
- **Named Entity Recognition (NER):** Identifying and classifying entities such as names of people, locations, dates, etc., within text.
- **Syntactic Parsing:** Understanding the grammatical structure and relationships of words in sentences.
- **Semantic Analysis:** Extracting the meaning of words and phrases from the text to understand the intent behind the input.

In addition to these features, NLTK supports several NLP tasks such as text classification, stemming and lemmatization, stopword removal, sentiment analysis, and question answering. This versatility makes NLTK an ideal choice for research and educational purposes, where customization and experimentation are needed.[7]

4.5.2 SpaCy

SpaCy is an open-source, high-performance library for NLP in Python. Designed specifically for production use, SpaCy excels in processing large volumes of text quickly and efficiently. While NLTK is commonly used for research, SpaCy is optimized for real-world applications, where speed, accuracy, and scalability are paramount.

Key features of SpaCy include [18]:

- **Tokenization:** Splitting text into individual tokens for easier manipulation.
- **Part-of-Speech Tagging:** Identifying the grammatical function of each word in a sentence.
- **Named Entity Recognition:** Extracting and categorizing entities within the text.
- **Syntactic Parsing:** Analyzing sentence structure to identify the relationship between words.
- **Semantic Analysis:** Understanding the meaning of words in context.

SpaCy comes with pre-trained models for multiple languages, offering support for a wide variety of NLP tasks. Additionally, it allows users to train custom models for specific use cases, making it highly extensible. The library's architecture is optimized for efficiency, with the Doc object centralizing the tokenized text and annotations, and the Vocab object managing word vectors and other lexical attributes, which help save memory.[18]

Summary
Both NLTK and SpaCy are essential tools in NLP, with distinct advantages depending on the project requirements. While NLTK offers deep customization for research-oriented projects, SpaCy's performance and ease of integration make it the ideal choice for real-world applications, such as AI assistants. By leveraging these tools effectively, it becomes possible to develop systems that understand, process, and generate human language, ultimately enhancing the performance of AI-driven solutions.
In the context of developing an AI assistant, integrating SpaCy for high-performance text processing alongside NLTK for specialized tasks can offer a powerful combination for achieving optimal language comprehension and interaction.

5 Feature Engineering

Feature engineering, is a critical step in the process of converting raw data into features that can be effectively used to train machine learning models. Features refer to the input data attributes that the model will use to learn patterns and make predictions. In its raw form, the data often cannot be directly fed into a machine learning model, as it may be incomplete, noisy, or not structured in a way that is conducive to learning. Therefore, feature engineering is necessary to refine the data before it can be used as input to the model.

The success of machine learning models is closely tied to the quality of the features used during training. Feature engineering involves a variety of techniques aimed at transforming or creating new features from existing ones. These techniques help highlight significant patterns and relationships in the data, which enable the machine learning model to learn more effectively and make accurate predictions. In essence, the goal is to enhance the quality and relevance of the features so that the model can perform optimally.

Feature engineering includes several important steps, such as:

- **Scaling or Normalizing Data:** Ensuring that the data is on a comparable scale, which can help certain models perform better by preventing the dominance of features due to their magnitude.
- **Encoding Non-Numeric Data:** Transforming categorical data, such as text or images, into numeric formats that machine learning algorithms can understand (e.g., one-hot encoding or label encoding).
- **Aggregating Data:** Summarizing or grouping data by time, entity, or other meaningful categories to extract higher-level insights (e.g., daily averages, sums, or counts).
- **Joining Data from Different Sources:** Combining datasets from multiple sources to enrich the feature set and provide more comprehensive information for the model.
- **Transferring Knowledge from Other Models:** Incorporating features or knowledge learned from other models to improve the current model's performance.

The ultimate objective of feature engineering is to maximize the machine learning algorithm's ability to learn from the dataset, thereby improving the model's predictive accuracy. By refining the dataset and focusing on relevant and meaningful features, the model becomes better equipped to make reliable predictions based on the data.

5.1 Feature Selection Techniques

Feature selection is a key step in building efficient machine learning models. It involves choosing the most relevant features from the dataset to improve the model's performance, reduce overfitting, and decrease computational complexity. Feature selection methods can broadly be divided into two categories: Automated Feature Selection using ML Libraries and Custom Feature Selection. Each method has its advantages, depending on the nature of the task and the data.

5.1.1 Machine Learning-Based Automated Feature Selection

Automated feature selection uses machine learning libraries to automatically choose the most relevant features for the model. These libraries include various algorithms designed to handle different types of data, particularly text data. Examples of such algorithms include:

- **Count Vectorizer:** This method counts the frequency of each word or token in the dataset, transforming the text into a numerical format. It is commonly used in natural language processing tasks.
- **Bag of Words (BoW):** A text representation method that converts text data into a set of word frequencies, disregarding grammar and word order.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** This technique assigns weights to words based on their importance within the entire dataset. Words that appear frequently in a document but infrequently across documents are considered more relevant.

These algorithms are particularly useful in processing text data and perform operations like stop word removal, tokenization, vectorization, and feature selection. The feature selection in these algorithms is often based on heuristics, such as word frequency or inverse document frequency. While these methods are generally effective for many use cases, they may not always produce the most intuitive or domain-specific features, especially for tasks requiring specialized knowledge.

In cases where the data is complex or the task requires domain expertise, custom feature selection becomes more necessary.

5.1.2 Custom Feature Engineering

Custom feature engineering is a more tailored and hands-on approach, where features are engineered specifically for the task at hand using domain knowledge. This process can be time-consuming and complex, but it is essential when domain-specific insights are required. For tasks that involve specialized data, such as medical diagnoses in psychiatry, custom feature engineering is often the better approach.

Custom feature engineering involves multiple steps, such as:

- **Text Parsing:** Extracting meaningful information from raw text data. This could include extracting symptoms, diagnoses, or treatments from patient records.
- **Stemming and Lemmatization:** These are processes that reduce words to their base or root forms. For example, "running" may be reduced to "run," and "better" may be reduced to "good."
- **Stop Word Removal:** Words like "the," "is," or "and" that do not carry significant meaning for the model are often removed from the dataset.
- **Exploratory Data Analysis:** Understanding the distribution of the data, identifying outliers, and identifying key patterns or correlations between variables.
- **Feature Selection:** Identifying the most relevant features for the task based on domain expertise and statistical methods.

While custom feature engineering is more resource-intensive, it often results in more accurate and domain-relevant features, which are critical for tasks like patient diagnosis in psychiatry. For this reason, in the current work, custom feature engineering methods are employed to select the most appropriate features for training the model. This approach ensures that the selected features accurately reflect the underlying relationships in the data and improve the model's ability to make accurate predictions.

By combining domain expertise with the right feature selection methods, the machine learning model can be fine-tuned to deliver more reliable and meaningful predictions.

5.2 Word Frequency Analysis

Word frequency analysis is a key technique in NLP that involves counting the occurrences of individual words within a text or corpus. This analysis provides insights into the frequency distribution of words, allowing for a deeper understanding of the text's content and structure. Word frequency analysis is often used to identify the most commonly occurring words, which can highlight key themes or topics present in the text.

To gain a more comprehensive understanding of the data and improve feature engineering, we expanded our analysis to not only focus on individual words but also on bigrams and trigrams. By examining these multi-word combinations, we can capture the context and relationships between adjacent words, providing a richer representation of the text compared to relying solely on individual words.

5.2.1 N-grams

N-grams are sequences of continuous words, symbols, or tokens within a document. In the context of NLP, n-grams represent the adjacent groupings of items in a text

and are crucial when analyzing textual data. They capture not only individual words but also the relationships between words in proximity, allowing for a more nuanced interpretation of the text.

The term "n" in n-grams refers to a positive integer that specifies the length of the sequence, which can take values such as 1, 2, 3, 4, and so on. Based on the value of n, n-grams can be categorized as follows:

1. Unigram: A sequence consisting of a single word.
2. Bigram: A sequence of two adjacent words.
3. Trigram: A sequence of three adjacent words.
4. N-gram: A general term that refers to sequences of n adjacent words, where n can be any positive integer.

By analyzing bigrams and trigrams, we can capture more detailed contextual information than with unigrams, as they provide insight into how words are related to each other within the text. This enables the identification of more meaningful patterns and relationships that can be used in feature engineering and predictive modeling.

5.2.2 Unigrams

Unigrams are the most basic form of n-grams, where n = 1. In this case, they represent individual words or tokens in the text. By analyzing unigrams, we can extract individual words that frequently appear across the dataset, providing an overview of the most common terms in the text.

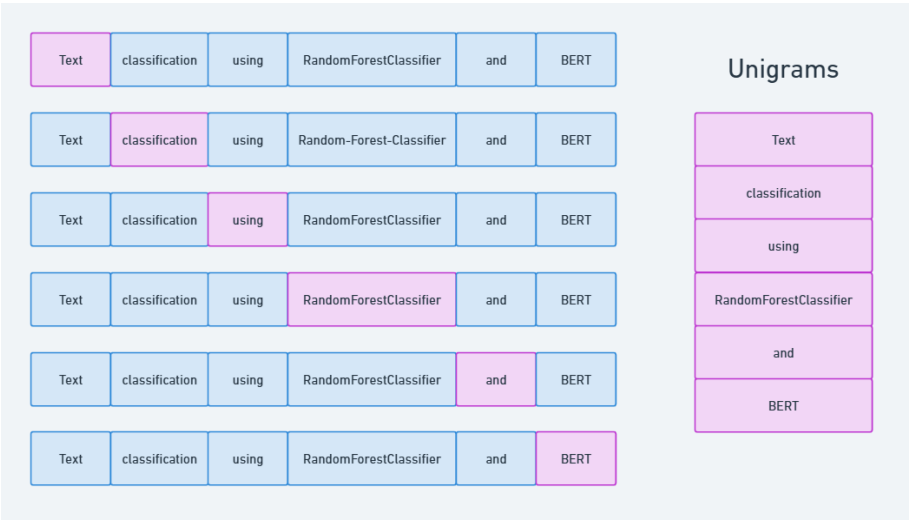

Figure 5.1: Unigrams

5.2.3 Bigrams

Bigrams are sequences of two adjacent words or tokens in a text. They represent a type of n-gram where n = 2, providing more context than unigrams by capturing pairs of words that often appear together. Bigram analysis is particularly useful for identifying common word combinations or collocations, offering a deeper understanding of the relationships between terms.

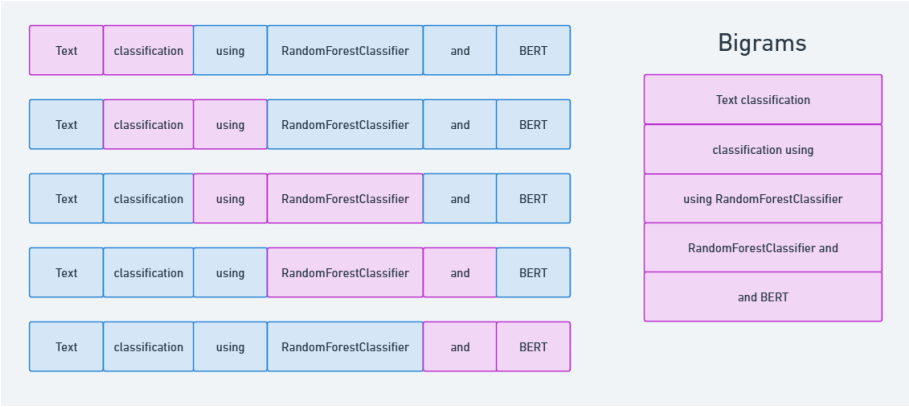

Figure 5.2: Bigrams

5.2.4 Trigrams

Trigrams are sequences of three adjacent words or tokens in a text. As a type of n-gram where n = 3, trigrams provide an even greater level of context than unigrams or bigrams. They capture frequent word triplets, which can help identify multi-word expressions or phrases that commonly appear together.

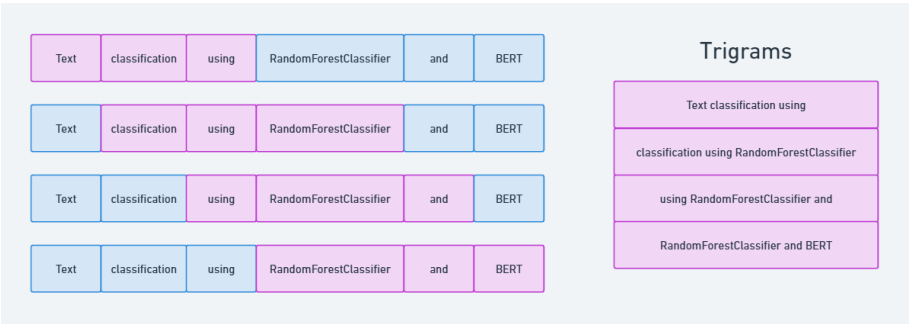

Figure 5.3: Trigrams

5.2.5 Implementation

To perform a more meaningful frequency analysis, it is crucial to combine the data from the multiple fields into a single column. This step simplifies the analysis process, as evaluating frequency for each of the 31 fields individually would be a tedious

and time-consuming task. Additionally, machine learning models typically require text data in a single column, rather than multiple fields. Therefore, concatenating all the text fields into one is essential for further processing.

We implemented a Python function to concatenate the data into a single column within the DataFrame. This function outputs a new column containing the combined text data from all fields. Afterward, we applied lowercasing to standardize the text, ensuring that case differences (e.g., "The" and "the") do not affect the analysis.

With the data preprocessed, we then proceeded to perform the word frequency analysis, considering not only unigrams, but also bigrams and trigrams. To calculate the frequency distributions, we utilized the NLTK's frequency distribution function.

5.2.6 Key Insights

After conducting the frequency analysis, we found that unigrams provided the most valuable insights, as they highlighted the most frequent and significant words in the text. Bigrams also proved useful and were incorporated into later stages of feature engineering. However, trigrams did not contribute additional useful information beyond what was captured by the bigrams, so we decided to omit them from further analysis.

To further refine our features, we performed separate frequency analyses for the psychiatric and non-psychiatric subsets of the data. This distinction allowed us to better understand the patterns specific to each category. The top 20 most frequent unigrams and bigrams for both the psychiatric and non-psychiatric datasets are shown below, providing a clearer view of the most important terms and their contextual relationships within each category.

After performing the word frequency analysis, we prepared two data frames, each containing the top 500 most frequent words from the psychiatric and non-psychiatric datasets. Upon examining these data frames, several key observations were made, which highlighted the need for further refinement and processing of the text data:

1. **Unexpected Word Associations:** One of the key observations was the presence of the word "schmerzen" (meaning "pain") in both data frames, but with differing frequencies. It was anticipated that this word would be primarily linked to the psychiatric cases, yet it was not. This discrepancy indicated that the context in which the word was used did not align with expectations, suggesting that further analysis was needed to better understand the nuances in how terms like "pain" were associated with the categories.
2. **Non-Standardized Data Entry:** The text data exhibited numerous instances of duplicated or fragmented entries due to non-standardized data entry practices. For example, the word "schmerzen" had a notably high frequency, but it did not fully represent the concept of pain in its various forms (e.g., head pain, neck pain, back pain, etc.). This issue was partly attributed to incomplete or incorrect data entry. In some cases, terms like "bauchschmerzen" (abdominal pain) were incorrectly split into two separate words, "bauch" (belly) and "schmerzen" (pain), skewing the frequency analysis.

3. **Synonym Usage:** The text, written by different personnel, contained numerous synonyms, which presented challenges in analysis. For example, words like "Bier," "vodka," and "alcohol" all referred to the same context of alcohol intoxication but were not consistently grouped together. This variability made it difficult to assess the true frequency of alcohol-related terms and hindered the clarity of certain contexts.

4. **Spelling Mistakes and Typos:** Despite the text processing steps, many spelling mistakes and typos remained unresolved. For instance, the words "alcohol" and "alkohol" were used interchangeably, but these variations were not corrected during the preprocessing. Such inconsistencies resulted in further noise in the dataset, complicating the analysis.

These observations underscored the need for additional refinement and preprocessing steps to improve the feature set and ensure the analysis accurately reflects the data's intended context. The issues identified, such as duplicated entries, synonym variation, and typos, highlighted gaps in data quality that must be addressed before meaningful insights could be derived.

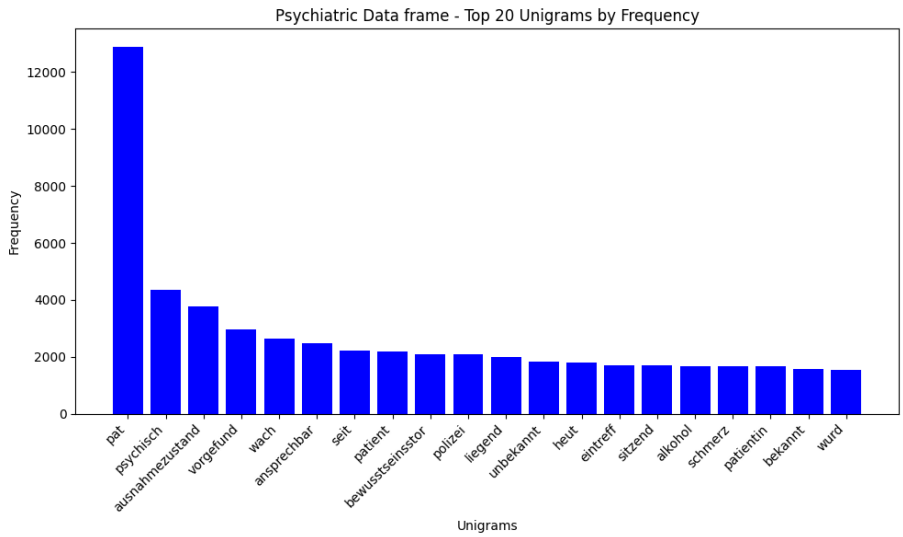

Figure 5.4: Top 20 Unigrams-Psychiatric Data Frame.

As a result of these challenges, we proceeded with further text analysis and processing strategies to improve the quality of the features extracted. These strategies, along with their implementation, are discussed in the following sections.

In the interim, we compiled a temporary vocabulary list based on the word frequency distribution. This list included words that had high occurrence frequencies in either of the two data frames (psychiatric or non-psychiatric). This preliminary vocabulary list will be refined and expanded as we apply additional text processing techniques in the subsequent steps.

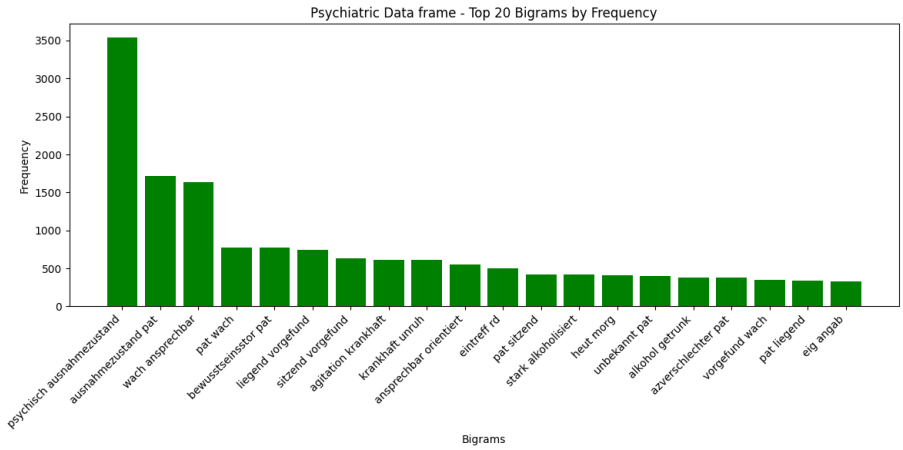

Figure 5.5: Top 20 Bigrams-Psychiatric Data Frame.

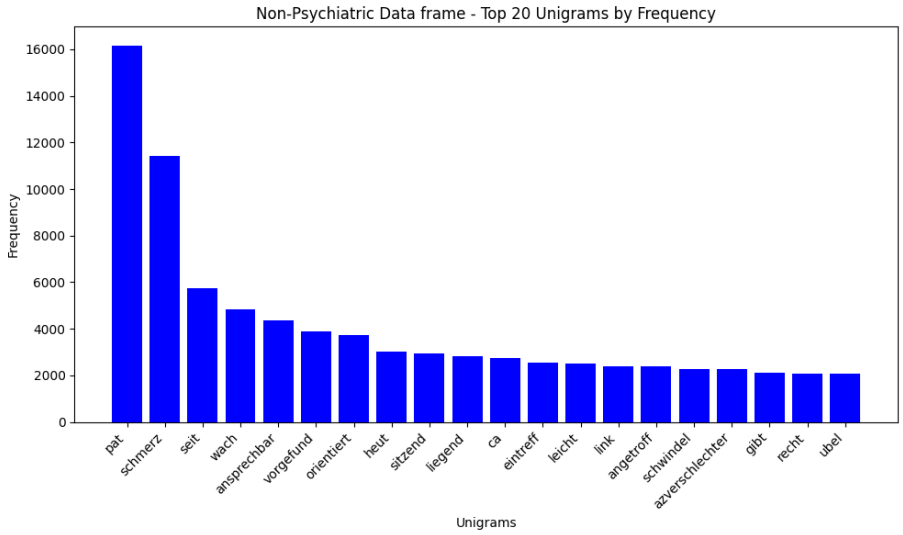

Figure 5.6: Top 20 Unigrams-Non-psychiatric Data Frame.

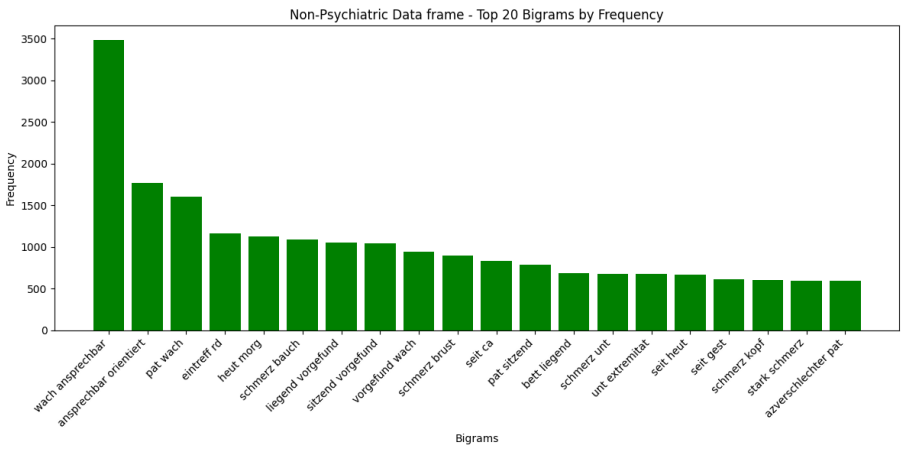

Figure 5.7: Top 20 Bigrams-Non-psychiatric Data Frame.

5.3 Comparative Word Frequency Distribution

One of the key insights obtained from the word frequency analysis was the presence of certain high-frequency words in both the psychiatric and non-psychiatric data frames. This raised a challenge in terms of associating these words with a specific label, or determining if they had any meaningful association with the labels at all. To address this ambiguity and better understand the relationship between the vocabulary list and the labels, we calculated the Comparative Word Frequency Distribution.

In the comparative word frequency distribution, we compute the relative frequency of each word in relation to the labels. In simple terms, this involves calculating the difference in the frequency of occurrence of each word between the psychiatric and non-psychiatric data frames. The difference in frequency values allows us to assess the strength and direction of association between the words and the labels. A positive difference indicates a stronger association with the psychiatric label, while a negative difference suggests a stronger association with the non-psychiatric label.

To implement this, we utilized the frequency data already calculated for the most frequent words in both the psychiatric and non-psychiatric data frames. The frequency values of each word in the vocabulary list were extracted from these frames. These frequency values were then stored in two separate data frames, one for psychiatric and one for non-psychiatric data. Using these data frames, we calculated the frequency difference for each word. Finally, we visualized the results by plotting the frequency differences on a graph, using the matplotlib library in Python.

The resulting Comparative Word Frequency Distribution graph provides a clear illustration of the relative association of each word with the two categories (psychiatric and non-psychiatric). The final graph, which is shown below, helps in understanding how strongly each word is linked to the psychiatric or non-psychiatric label, and informs the next steps in refining feature extraction for machine learning models.

The graph illustrates the frequency distribution of the vocabulary list between the psychiatric and non-psychiatric labels. On the Y-axis, we have the vocabulary list, while the X-axis represents the frequency difference between the two labels. The frequency difference is calculated as the difference between the frequency of each word in the psychiatric data frame (label 1) and the non-psychiatric data frame (label 0). Consequently, negative values on the X-axis indicate that the word is more strongly associated with label 0 (non-psychiatric), while positive values indicate a stronger association with label 1 (psychiatric).

Based on this analysis, we reduced the vocabulary list to the top 10 most meaningful words. A notable observation can be made from the graph, particularly regarding the words ‘kopf’, ‘brust’, ‘extremitaten’, ‘kopfschmerzen’, and ‘schmerzen’. These words are related to the concept of pain, but not all of them provide complete and useful information on their own. While ‘kopfschmerzen’ (headache) conveys clear medical information, the words ‘kopf’ (head), ‘brust’ (chest), ‘extremitaten’ (limbs), and ‘schmerzen’ (pain) on their own are less informative. These individual words require contextual pairing to deliver meaningful insights. For instance, ‘kopf’ and

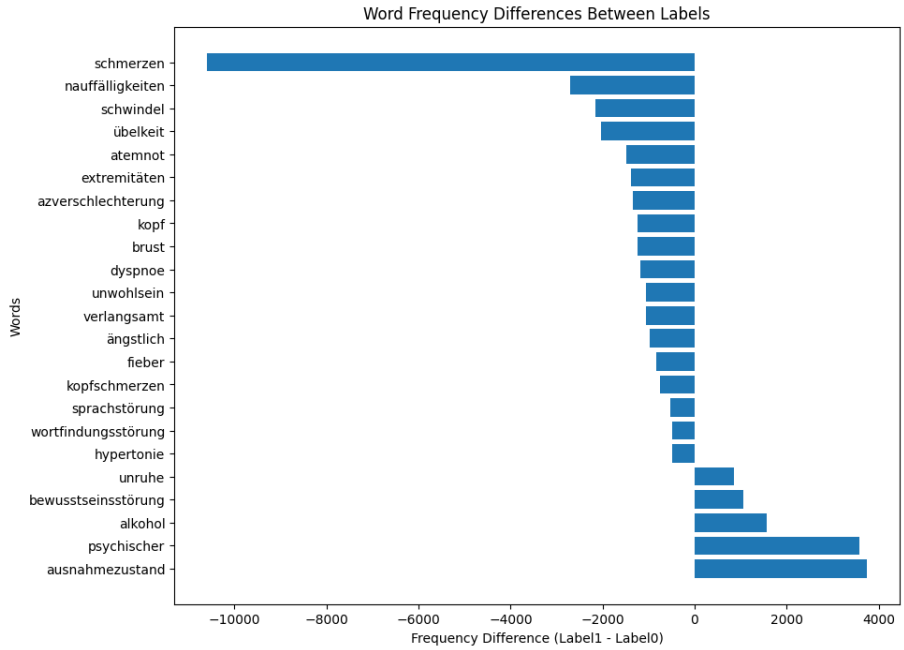

Figure 5.8: Word frequency distribution graph

'brust' need to be paired with 'schmerzen' (pain) to provide more specific information about the location of pain.
To improve the feature set, we propose combining these words where appropriate. Specifically, pairing 'kopf' with 'schmerzen' (headache), and 'brust' with 'schmerzen' (chest pain), among others, would yield more valuable features for classification. This approach will be further explored and addressed in the following sections.

5.4 Contextual Text Analysis

After performing word frequency analysis for unigrams, bigrams, and trigrams, we initially considered only unigrams for feature engineering. However, now we will integrate bigrams into the process, which offer more context by considering pairs of consecutive words. As mentioned, we have already identified the most frequently occurring bigrams in both the psychiatric and non-psychiatric datasets. To decide which words should be combined into a single feature, we must refer to the frequency distribution of bigrams.

There are two potential ways to handle combining words into features:

1. Bigrams as Features
2. Composite Features

5.4.1 Bigrams as Features

In this approach, we consider frequently occurring bigrams as new features. Bigrams represent pairs of words that appear consecutively in the text. Since we have

identified the most common bigrams in both datasets, we can examine their relative frequencies across labels. Bigrams with a significant positive or negative frequency difference between labels can be treated as good candidates for new features.

However, we must be cautious because some bigrams may appear frequently but do not add much new information. For example:

- 'heute morgen' (today morning) may have a high frequency, but it is simply a time indicator that doesn't provide much meaningful insight into the context of the text.
- 'schmerz bauch' (stomach pain) might also be highly frequent, but it can be reduced to a single word, 'bauchschmerzen' (stomach pain), which already conveys the full meaning.

In such cases, bigrams may not contribute valuable new information. Therefore, we need a separate function to identify and combine such words, which will be discussed in the next section.

5.4.2 Composite Features

This method aims to combine words that, when grouped, provide meaningful information. To achieve this, we defined a custom Python function to identify words that frequently appear before and after a target word.

Functionality of the Python function:

- The function takes a target word as input.
- It scans the entire dataset for occurrences of this target word.
- For each occurrence, it checks the surrounding words (those occurring before and after the target word).
- The function counts how often these combinations appear together.
- Finally, it outputs a list of surrounding words and their frequency of co-occurrence with the target word.

This statistical data is crucial because it helps us identify which words commonly pair with the target word. Based on the frequency of these word pairs, we can decide which combinations should be treated as composite features.

Example with the word 'schmerzen' (pain):

- The word 'schmerzen' was frequently paired with words like 'bauch' (stomach), 'brust' (chest), 'hand', 'kopf' (head), etc.
- By analyzing these combinations, we found that 'schmerzen' (pain) is more informative when paired with words that specify the body part or location of pain (e.g., 'bauchschmerzen' (stomach pain), 'kopfschmerzen' (headache)).
- Alone, 'schmerzen' doesn't convey enough information about the patient's suffering, but the combination of 'schmerzen' with these other words offers a much richer context.

By applying this function across the entire vocabulary list, we refined the feature set, merging meaningful combinations of words into composite features. This step significantly reduced the number of features, ensuring that only the most informative combinations were retained for further analysis.

5.5 Vocabulary Synonym Mapping

In previous sections, we have focused on applying statistical methods for feature engineering. However, it is equally important to address the text from a grammatical perspective. As mentioned earlier, the data used in this project was not recorded in a consistent language format; it was written by different individuals, using varying tones and voices. Consequently, during the statistical analysis, we observed that the presence of synonyms alongside key vocabulary words was distorting the frequency distribution of the vocabulary list. To enhance the robustness of the model, it is necessary to consider both synonyms and common spelling errors or typos. This will ensure that the model can recognize synonyms when processing new patient data, leading to more accurate predictions.

To achieve this, we employed the Levenshtein distance method, which is commonly used in linguistics, information theory, and computer science to measure the difference between two sequences. The Levenshtein distance quantifies the minimum number of single-character edits (insertions, deletions, or substitutions) required to transform one word into another. This method, introduced by Soviet mathematician Vladimir Levenshtein in 1965, allows for the comparison of words based on their similarity.[19]

Using the Levenshtein distance metric, we developed a custom Python function to identify words like those in the vocabulary list. The function, named is_similar(), compares two words based on the Levenshtein distance and determines whether they are sufficiently similar. The similarity threshold can be adjusted, allowing for flexibility in how much distance is tolerated before two words are classified as similar. The threshold can range from 0 (for exact matches) to the length of the longer word (which represents the maximum number of edits needed to convert two completely different words into the same word).

By applying this method, we successfully mapped many synonyms present in the text to their corresponding words in the vocabulary list. This mapping process was recorded in a dictionary, which will be utilized in the later stages of the project to ensure accurate and consistent predictions across various data inputs.

5.6 Recursive Feature Elimination and Cross Validation (RFECV)

Upon refining the temporary vocabulary list through the previously discussed feature engineering and selection processes, we arrive at the final stage of feature engineering: Recursive Feature Elimination and Cross Validation (RFECV).

Recursive Feature Elimination (RFE) is a powerful feature selection technique used to identify and retain the most relevant features in a dataset. RFE works iteratively by starting with all features and progressively eliminating the least important ones until the desired subset of features is achieved. The underlying principle of RFE is that the most relevant features are those that have the greatest impact on the target variable, making them most useful for prediction. To assess the importance of each feature, RFE uses a model, such as linear regression or support vector machine (SVM), and eliminates features with the lowest importance score in each iteration[5].

For our project, we utilized the RFECV algorithm from the scikit-learn library and employed a Support Vector Machine (SVM) as the estimator. This allowed us to further refine the feature list by selecting the 10 most significant features. The final set of features, which will be used for prediction in the model, is as follows:

Vocabulary: ['psychisch', 'ausnahmezustand', 'alkohol', 'azverschlechter', 'unruh', 'suizidal', 'schwindel', 'fieb', 'pain_related', 'atemnot']

These features, selected through a combination of RFE and cross-validation, form the core of our feature set, capturing the most relevant information for classification tasks.

5.7 Training Dataset

Having completed the feature engineering process, the next step is to prepare the final training dataset for the machine learning model. At the end of the previous section, we had a cleaned and preprocessed combined dataset. However, before we can send this dataset to the machine learning algorithm, a few more processing steps are required. This section outlines the steps for preparing the final training dataset.

5.7.1 Mappings

In previous sections, we identified synonyms in the text that correspond to words in the vocabulary list. Since synonyms convey the same information, but the machine learning algorithm does not inherently understand this, each synonym would be counted as a separate entity, potentially skewing the results. To address this, we need to map and replace all synonyms in the text with their corresponding vocabulary words.

We have already created a dictionary containing the mappings for the vocabulary list. Using the built-in replace() method in Python, we will replace the synonyms in the text according to this mapping. A for loop will be used to perform these replacements, ensuring that all synonyms are replaced with the correct vocabulary words.

Additionally, as discussed in previous sections, some words, when combined, provide more meaningful information and need to be treated as composite features. For

example, words related to pain, such as "bein," "bauch," "kopf," and "brust," need to be combined into a single composite feature named "pain_related."

5.7.2 Text and Target Separation

The final step in dataset preparation is to separate the text data from the target labels. This is essential because the machine learning model requires the text and target data to be stored separately for vectorization. The vectorizer will convert the text data into a numeric form, which will be used for training the model.

We will store the text data in a data frame named text_column and the target labels in a separate data frame named target. This separation will ensure that the machine learning model can process both the input features and their corresponding labels appropriately.

With this, the dataset is fully prepared for machine learning model training. In the next section, we will explore the machine learning models and the training process.

6 Model Selection and Training

The goal of this project is to develop an AI assistant to support emergency rescue personnel in diagnosing psychiatric patients. Achieving this objective requires not only robust data and effective data processing but also the selection of appropriate machine learning models suited to the task.

A suitable machine learning model for text classification should possess the following qualities:

1. **High Accuracy:** The model should correctly classify a large proportion of input texts.
2. **Strong F1 Score:** A high F1 score, combining precision and recall, indicates balanced and strong performance in classification tasks.
3. **Good Generalization:** The model should be able to perform well on unseen data, not just on the training set, ensuring it is not overfitting.
4. **Efficient Computation:** For real-world applications, the model should be computationally efficient, capable of processing large volumes of text data quickly.
5. **Effective Feature Engineering:** The model should leverage well-engineered features that capture relevant information from the input text data.
6. **Robust Preprocessing:** The model should incorporate thorough text preprocessing, including tasks like cleaning, normalization, and standardization of text data.
7. **Adaptability:** The model should handle variations in text input and potentially be fine-tuned for specific domains or tasks, making it versatile.
8. **Ability to Handle Imbalanced Data:** The model should be robust enough to perform well even in cases where one class (e.g., psychiatric vs. non-psychiatric) is more prevalent than the other.

Upon careful evaluation, the Random Forest model was selected as the primary choice due to its ability to meet the above criteria. The Random Forest algorithm is an ensemble model that excels in classification tasks, is computationally efficient, and handles feature interactions well, making it a strong candidate for this project.

However, recognizing the complexity and critical nature of diagnosing psychiatric conditions, we also decided to leverage advanced algorithms based on the Transformer architecture, specifically BERT. Unlike Random Forest, BERT excels at understanding the nuances of language structure, grammar, and context. BERT has a deeper comprehension of text and can capture contextual relationships in the data,

which makes it particularly effective for more complex natural language processing tasks.

By combining the strengths of the Random Forest model and BERT, we aim to build a powerful and accurate AI assistant capable of supporting emergency rescue personnel in diagnosing psychiatric patients.

6.1 Random Forest Classifier

The Random Forest algorithm is an ensemble learning method that combines multiple decision trees to enhance classification accuracy. Each decision tree in the forest is trained on a different subset of the training data, known as the bootstrap sample (a sample drawn with replacement). This randomness helps reduce overfitting and enhances model generalization [15].

One-third of the data, not used in the training set, is set aside as the out-of-bag (oob) sample, which is utilized for model evaluation, providing an additional layer of validation. Furthermore, feature bagging introduces more randomness by selecting different subsets of features for each tree, reducing the correlation between trees and improving model diversity.

In classification tasks, predictions are made through a majority vote, where the class that appears most frequently across all decision trees becomes the final prediction. For regression tasks, the average of the individual tree predictions is used.

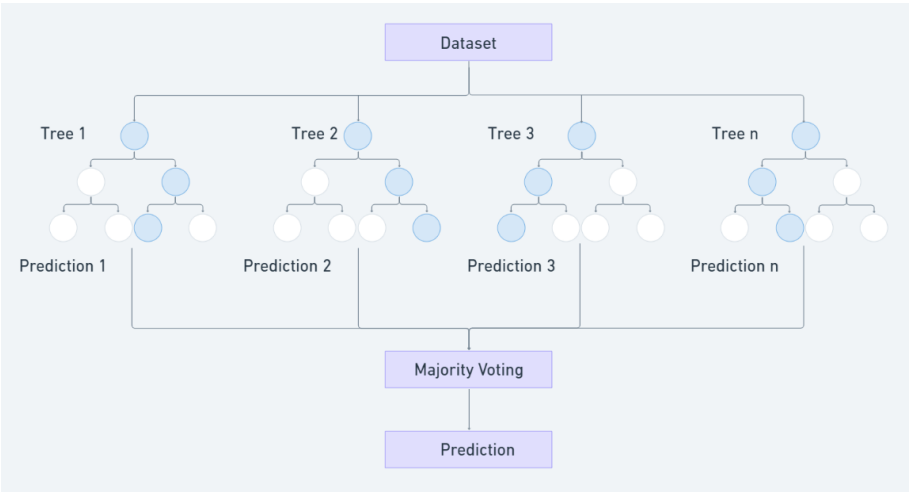

Figure 6.1: Random Forest Architecture

There are three key hyperparameters in the Random Forest algorithm that need to be set prior to training:

- **Node size:** Determines the minimum number of samples required to split an internal node.
- **Number of trees:** Specifies how many decision trees are built in the forest.

- **Number of features sampled:** Specifies how many features are randomly selected for each tree.

These hyperparameters influence the model's performance, and proper tuning is essential for achieving optimal results.

6.2 Bidirectional Encoder Representations from Transformers (BERT)

BERT is an open-source machine learning framework developed for NLP tasks. Unlike traditional language models that process text sequentially, BERT is designed to read text bidirectionally—simultaneously from left to right and right to left. This bidirectional reading allows BERT to better understand the context and meaning of words in ambiguous or complex sentences [10].

BERT is based on the Transformer model, which uses self-attention mechanisms to connect every output element to every input element, dynamically calculating the weights of these connections based on their relationships within the text. This allows the model to capture long-range dependencies and contextual information across sentences.

Initially, BERT was pre-trained using vast text datasets from Wikipedia and Books Corpus, and it can be fine-tuned for specific tasks, such as question answering, sentiment analysis, or classification, using task-specific datasets. This fine-tuning process adjusts the model's parameters to improve performance on the given task. BERT has revolutionized the way machines understand and process language, making it particularly effective for tasks requiring deep contextual understanding.

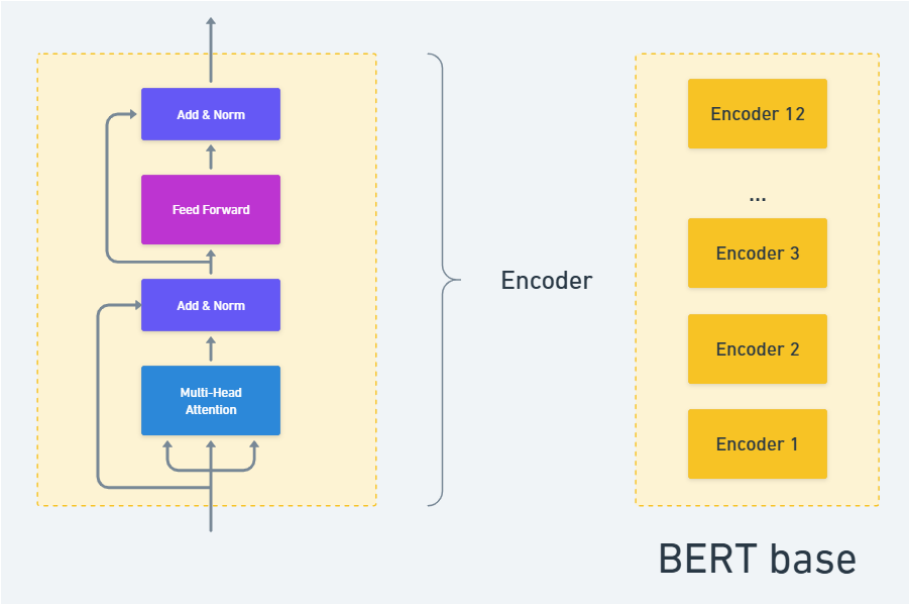

Figure 6.2: BERT Architecture

6.3 Model Training

Model training refers to the process of teaching a machine learning algorithm to make predictions or decisions by exposing it to data. The key steps involved in the training process are outlined below:

- **Data Preparation:** Before training the model, the dataset must be split into distinct sets: training, validation, and test sets. A typical split could be 70/15/15 or 80/10/10, where most of the data (70–80%) is used for training the model, 10–15% is reserved for validation during training, and the remaining 10–15% is used to evaluate the model's final performance after training.
- **Algorithm Selection:** The choice of algorithm depends on the type of problem being solved. For instance, in classification tasks, models like Random Forest, SVM, or BERT may be chosen, while for regression tasks, linear regression or decision trees could be more appropriate. The selected algorithm should align with the problem's requirements, whether it's classification, regression, or clustering.
- **Training Process:** The model is trained by feeding it the input data and allowing it to learn patterns, relationships, and structures from the data. During training, the model adjusts its internal parameters to best capture the patterns in the data.
- **Optimization:** To improve the model's performance, optimization techniques are applied. For example, gradient descent is commonly used to minimize errors by adjusting the model's weights iteratively. This process helps the model converge towards an optimal set of parameters, which enhances its predictive accuracy.
- **Evaluation:** After the model is trained, its performance is evaluated using appropriate metrics such as accuracy, precision, recall, F1 score, or AUC-ROC, depending on the nature of the problem and the specific objectives. Evaluation metrics help in understanding how well the model generalizes to unseen data.
- **Iteration:** Model training is typically an iterative process. After evaluating the model, the algorithm may be refined through multiple cycles of retraining, tweaking parameters, or modifying features. This iteration helps to improve the model's accuracy and robustness.
- **Hyperparameter Tuning:** In addition to learning from the data, models also have hyperparameters that can be tuned to optimize performance. These may include settings such as the learning rate, batch size, or the number of trees in Random Forest. Grid search or random search methods can be used to find the optimal combination of these hyperparameters, ensuring that the model performs at its best.

6.3.1 Random Forest Classifier

This section outlines the training process of the Random Forest classifier, following the key steps highlighted earlier.

6.3.1.1 Data preparation

As described in previous sections, the text data has been cleaned, preprocessed, and separated into text and label datasets to facilitate model training. However, the Random Forest algorithm cannot process raw text data directly, as it requires numerical inputs. Thus, the text data must be transformed into numerical representations using vectorization techniques.

1. Vectorization

Text vectorization is the process of converting textual information into numerical representations in a multidimensional space, enabling machines to analyze and process human language. This step bridges the gap between human language and machine understanding and is crucial for tasks such as:

- Text classification
- Sentiment analysis
- Information retrieval
- Semantic search

Various methods can be employed for text vectorization:

1. **Bag-of-Words:** Represents documents as vectors based on word frequency, without considering word order or context.
2. **TF-IDF:** Extends BoW by weighing word frequency against its importance across the dataset.
3. **Word Embeddings:** Techniques like Word2Vec or GloVe capture semantic relationships between words, producing dense vector representations.
4. **Transformer Models:** Advanced approaches, such as BERT, generate nuanced representations by incorporating contextual information.

2. Bag-of-Words Vectorization

In our implementation, we selected the Bag-of-Words approach as the vectorization method. The preprocessed text data, separated earlier, is now passed to the BoW algorithm for transformation into feature vectors.

The BoW algorithm processes the entire dataset to create a binary feature vector for each patient instance. It assigns a value of 1 to a feature if the feature is present in the patient's text and 0 otherwise. This process is repeated for all patient instances, resulting in a complete feature matrix for the dataset. The final output is a data frame with columns representing the vocabulary list (our predefined features) and

rows representing 24,153 patient instances. The following steps were followed in the implementation:

Step 1: Defining the Vectorizer

The CountVectorizer() function from scikit-learn was used as the vectorizer for creating feature vectors. The CountVectorizer() function includes a vocabulary argument that specifies the features to be considered.

- If this argument is left empty, the algorithm defaults to treating each unique word in the dataset as a feature. This can result in an extremely large vocabulary of tens of thousands of features, which is computationally inefficient.
- To overcome this, we used our custom-engineered feature list by setting vocabulary=Vocabulary, where Vocabulary is the Python list containing our predefined features. This ensures the vectorizer focuses only on our relevant features.

Step 2: Transforming Text Data into Vectors

After defining the vectorizer, the text data is passed through the CountVectorizer() algorithm. This converts the text into a numerical matrix, with rows representing patient instances and columns corresponding to the predefined features.

	psychisch	ausnahmezustand	alkohol	azverschlechter	unruh	suizidal	schwindel	fieb	pain_related	atemnot
0	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	2	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0
4	0	0	0	0	2	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...
24148	0	0	0	0	0	0	0	0	0	0
24149	0	0	0	1	0	0	0	0	0	0
24150	0	0	0	0	0	0	0	0	0	0
24151	0	0	0	0	0	0	0	0	0	0
24152	0	0	0	0	0	0	1	0	4	0

24153 rows × 10 columns

Figure 6.3: Vectorized Data Frame

Step 3: Converting Vectorized Data into a Data Frame

Finally, the vectorized data is converted into a structured data frame, with field names corresponding to the vocabulary list. This data frame serves as the input for training the Random Forest classifier. This structured approach ensures that the text data is efficiently transformed into a machine-readable format, enabling effective training of the Random Forest classifier.

6.3.1.2 Train-Test Split

With the vectorized data prepared, the next step is to split it into training and test sets. This is a fundamental practice in machine learning to ensure the model is trained effectively and evaluated fairly. Training the model on the entire dataset without splitting can lead to biased results and poor generalization to unseen data.

6.3.1.3 Training the Random Forest Model

With the training and testing datasets ready, we can now proceed to train the Random Forest model. The process involves importing the Random Forest classifier, setting its hyperparameters, and training the model using the fit() method.

Step 1: Instantiating the Random Forest Classifier

The RandomForestClassifier() is imported from scikit-learn's ensemble library. It serves as the model object that will later be trained on the data.

Hyperparameters Overview:

Hyperparameters are the parameters that need to be defined before training the model. They are not learned from the data but play a crucial role in controlling the model's behavior and performance.[4]

Some key hyperparameters for Random Forest Classifier include:

1. **n_estimators:** The number of trees in the forest. A larger number of trees can improve performance but increases computational cost.
2. **max_depth:** The maximum depth of the individual decision trees. Controlling this helps prevent overfitting.
3. **mi_samples_split:** The minimum number of samples required to split an internal node.
4. **min_samples_leaf:** The minimum number of samples required to be at a leaf node.
5. **criterion:** Specifies the function to measure the quality of a split. For classification, common options are gini and entropy.
6. **random_state:** Ensures reproducibility by controlling randomness in the model.

Hyperparameter Tuning:

Hyperparameters can significantly impact model performance. Tools like GridSearchCV and RandomizedSearchCV are often used for systematic tuning to find the optimal combination. However, care must be taken to avoid overfitting during this process [4].

Step 2: Training the Model

After defining and configuring the model, we train it using the fit() method. This method takes the training feature set (X_train) and the corresponding labels (y_train) as input. By following this structured approach, we ensure that the Random Forest model is trained effectively and optimized for the given application.

6.3.2 BERT

This section details the training process for the transformer-based model, BERT. Unlike the Random Forest classifier, BERT utilizes a transformer architecture, which

requires input data to be converted into numerical tensors. Here, we outline the dataset preparation process tailored for BERT and provide a detailed explanation of the required preprocessing steps and code implementation.

6.3.2.1 Dataset Preparation

Preparing the dataset for BERT involves tokenizing text data and converting it into tensors. This process is necessary as BERT models process data in the form of tokenized sequences.

The tokenize_data() function is a custom utility designed to preprocess and tokenize input text data for use with BERT. Its primary purpose is to convert raw text into a numerical format that BERT can understand while ensuring input consistency. The function plays a critical role in NLP pipelines, particularly in tasks involving transformer-based models like BERT. It standardizes and prepares input data, ensuring:

1. **Compatibility:** Converts raw text into a format BERT can process, including token IDs and attention masks.
2. **Efficiency:** Handles variable-length sequences by applying padding and truncation, allowing for streamlined batch processing.
3. **Integration:** Works seamlessly with PyTorch for training and inference, bridging the gap between preprocessing and model execution.

This function ensures that the input data adheres to the architectural requirements of BERT, enabling robust and consistent performance during model training and inference.

6.3.2.2 Defining the MedicalDataset Class

The next step in our pipeline is to define a custom dataset class called MedicalDataset. This class is specifically designed for handling tokenized medical text data and their associated labels. It inherits from the torch.utils.data.Dataset class, which is PyTorch's standard framework for working with datasets.

6.3.2.3 Train-Test Split

Now that the tokenize_data() function and the MedicalDataset class have been defined, we can prepare the training and evaluation datasets. The process involves several key steps to ensure the data is correctly tokenized, split, and formatted for use with BERT and PyTorch.

Step 1: Initializing the Model and Tokenizer

The first step is to load a pre-trained BERT model and its tokenizer from Hugging Face's transformers library. This ensures we have a foundation for fine-tuning the model on our specific task (binary classification).

Step 2: Splitting the Data into Training and Evaluation Sets

To evaluate the model's performance, we need to split the dataset into training and evaluation datasets.

Step 3: Tokenizing the Training and Evaluation Datasets

The text data for both training and evaluation datasets must be tokenized to convert it into a numerical format suitable for input into the model.

Step 4: Ensuring the Labels are Tensors

The labels must also be converted into PyTorch tensors to match the format of the tokenized inputs.

Step 5: Creating the Final Training and Evaluation Datasets

Using the MedicalDataset class, we create dataset objects for training and evaluation.

The next step involves defining the training parameters, evaluation metrics, and training the BERT model. This will be covered in the subsequent section.

6.3.2.4 Training and Evaluation

Now that we have discussed dataset preparation and tokenization, the next step involves setting the training and evaluation parameters before we begin training the BERT model.

Step 1: Setting Training Parameters

The TrainingArguments class from Hugging Face's transformers library is used to configure the training process. It defines the hyperparameters and settings required for fine-tuning the BERT model.

Step 2: Function to Set Evaluation Metrics

The compute_metrics() function is essential for evaluating the model's performance during fine-tuning. It calculates key evaluation metrics such as accuracy, precision, recall, and F1-score, and is called during the evaluation process after each epoch.

Step 3: Model Training

The Trainer class simplifies the training process by managing the training loop, evaluation, and logging. It allows us to define a single function to initialize and train the model.

Step 4: Model Training

Once the Trainer is initialized, the train() method is called to start the fine-tuning process.

Step 5: Evaluation and Logging

After each epoch, the model's performance is evaluated on the validation dataset, and key metrics are logged. These metrics provide insight into how well the model is generalizing.

Training Output:

1. **Training Loss:** Indicates how well the model is fitting the training data. A decrease in training loss indicates better fitting.
2. **Evaluation Metrics:** Accuracy, precision, recall, and F1-score values for each epoch, which provide an understanding of how well the model is performing on unseen data.
3. **Checkpoints:** The model is saved after every epoch for later use. This allows us to restore the model if training is interrupted or if we want to deploy the best-performing model.

7 Performance Evaluation

Now that both models (Random Forest Classifier and BERT) are trained, it is crucial to evaluate their performance before deployment. The evaluation process involves assessing how well each model predicts psychiatric and non-psychiatric classifications on unseen data. This will help determine the model's ability to generalize and its suitability for real-world applications.

7.1 Performance Evaluation: Random Forest Classifier

The performance of the Random Forest Classifier is evaluated on a test dataset, which contains unseen data that the model has not encountered during training. This will help us determine how well the model generalizes to new examples.

Steps for Performance Evaluation:

1. **Model Predictions:** First, we pass the test dataset through the trained Random Forest model and obtain predictions. The predictions can be generated using the predict() method of the Random Forest Classifier.
2. **Comparison of Predicted vs Actual Values:** We compare the predictions (y_pred) with the actual target values (y_test) from the test dataset. This comparison will give us an idea of how accurate the model is.
3. **Performance Metrics:** Next, we calculate various performance metrics to assess the model's effectiveness:
   - **Accuracy:** Measures the proportion of correctly predicted instances.
   - **Precision:** The proportion of positive predictions that are correct.
   - **Recall:** The proportion of actual positive instances that were correctly identified.
   - **F1-Score:** The harmonic mean of precision and recall, providing a balance between the two.
4. **Classification Report:** We use the classification_report() function from scikit-learn to generate a comprehensive report of these metrics.

Overall Model Performance:

- The accuracy of the model is 0.81, indicating that 81% of the total predictions are correct.

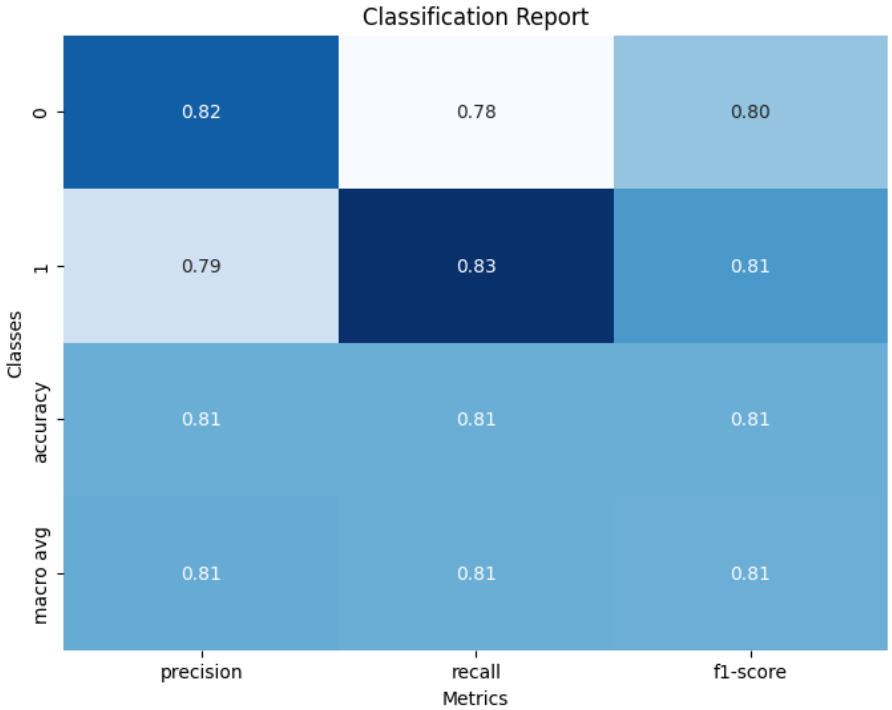

Figure 7.1: Classification Report:Random Forest Classifier

- The macro average of precision, recall, and F1-score is 0.80, meaning that both classes contribute equally to the overall performance.
- The weighted average is also 0.80, considering class imbalance while computing the average.

Interpretation and Implications

The classification report suggests that the Random Forest classifier achieves a balanced trade-off between precision and recall, as reflected in the high F1-scores for both classes. The model demonstrates robust performance in distinguishing between the two categories, making it a suitable choice for integration into the AI assistant After evaluating the Random Forest Classifier, we will follow similar steps to evaluate the BERT, comparing its performance with that of the Random Forest model. However, since BERT provides more nuanced contextual understanding, we expect it to potentially outperform the Random Forest model in terms of handling complex textual data.

7.2 Performance Evaluation: BERT

After fine-tuning the BERT model, the performance was evaluated based on the metrics defined during the training process. The trainer() function automatically generated an evaluation report, providing detailed insights into the model's performance across multiple epochs.

Evaluation Results of Fine-Tuned BERT Model:

1. **Training Learning Rate:** The learning rate started at a relatively higher value and gradually decayed to near zero by the end of the last epoch. This gradual decay is a standard strategy to fine-tune the model and avoid overshooting the optimal weights.
   - **Key Observation:** The learning rate decayed linearly throughout the training, allowing for smoother convergence as the model progressed through the epochs.

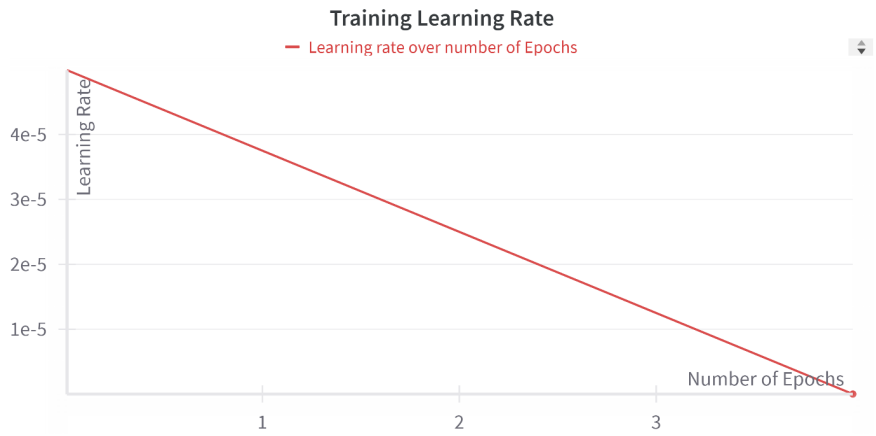

Figure 7.2: Learning rate:BERT

2. **Training Loss:** The training loss started at relatively high values (between 0.2 and 0.3) at the beginning of the training process. However, as the model fine-tuned, the loss dropped significantly, reaching close to 0.05 by the last epoch.
   - **Key Observation:** The decreasing training loss indicates that the model is improving and learning the patterns in the dataset effectively. The steep drop suggests that BERT was able to learn from the data quickly and optimize its parameters efficiently.
3. **Accuracy:** The accuracy chart shows the performance of the model on the evaluation set over the epochs. The accuracy fluctuated between 0.961 and 0.967 across the epochs, with an average accuracy of 96.4%.
   - **Key Observation:** The high and consistent accuracy across epochs demonstrates the model's strong ability to correctly classify psychiatric and non-psychiatric cases. The stable performance indicates the model's ability to generalize well to unseen data.
4. **Precision:** The precision chart shows the variation in prediction precision across epochs. The average precision achieved by the model was 97.5%.
   - **Key Observation:** A precision of 97.5% indicates that when the BERT model predicts a positive instance (psychiatric), it is very likely to be correct. High precision means that false positives are minimized, which is crucial for medical diagnoses.

Figure 7.3: Training loss:BERT

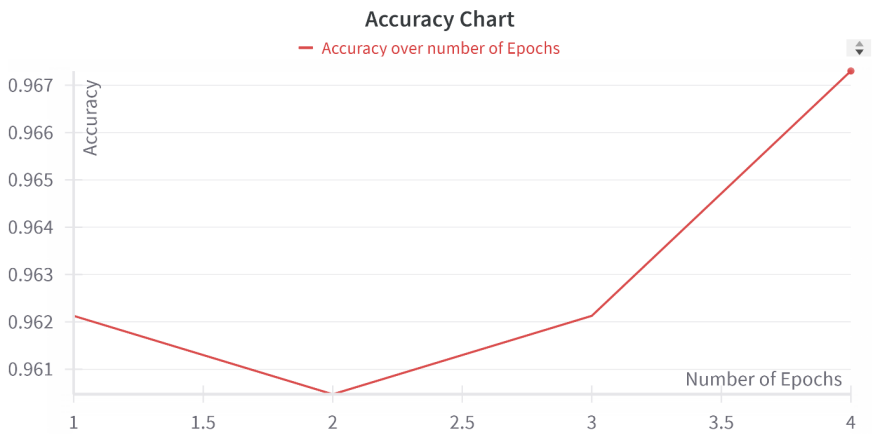

Figure 7.4: Accuracy:BERT

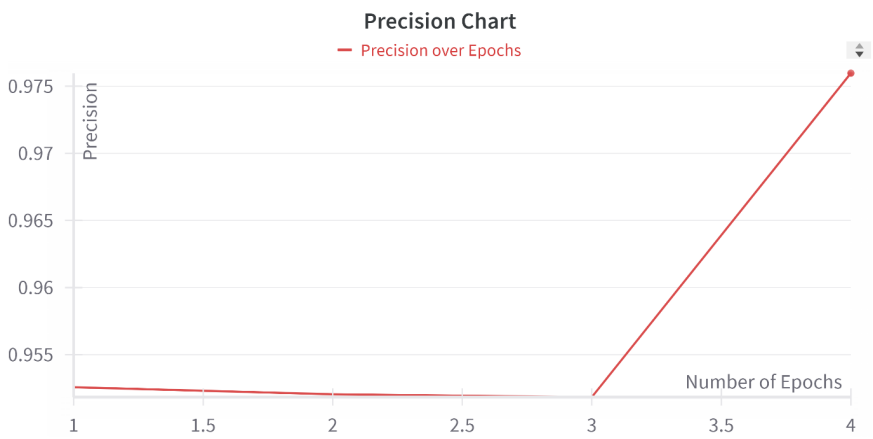

Figure 7.5: Precision:BERT

5. **Recall:** The recall chart depicts the model's ability to identify all relevant instances (actual psychiatric cases). Recall values varied between 0.972 and 0.96, with an average recall of 96.6%.
   - **Key Observation:** The recall score is high, meaning the model is effective at identifying most of the true psychiatric cases. High recall is important in a medical setting, as it ensures that most actual cases are correctly identified.

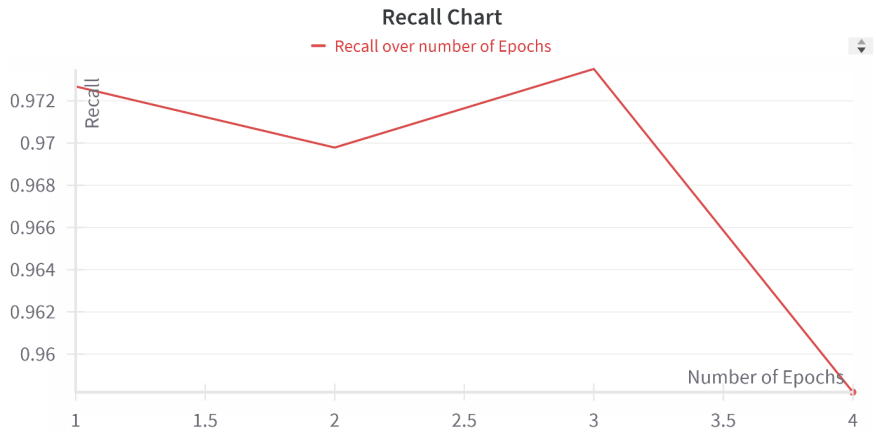

Figure 7.6: Recall:BERT

6. **F1 Score:** The F1 score represents the harmonic mean of precision and recall, providing a balance between the two metrics. The F1 score varied over the epochs, with an average F1 score of 96.4%.
   - **Key Observation:** The F1 score reflects the model's ability to balance precision and recall effectively, making it a reliable metric for overall model performance. An F1 score above 96% shows that the model is both precise and sensitive in predicting psychiatric cases.

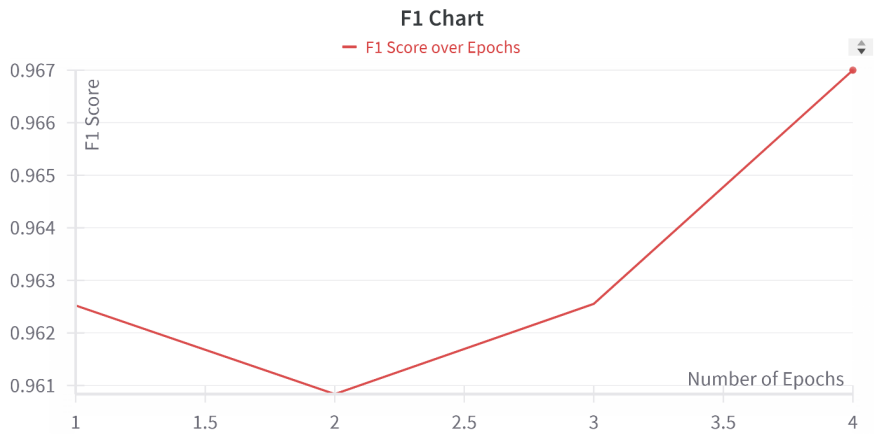

Figure 7.7: F1-Score:BERT

These results indicate that the BERT model has performed exceptionally well, with high accuracy, precision, recall, and F1 score across all epochs. This suggests that BERT is well-suited for this task and can be deployed for real-world psychiatric classification with confidence.

Comparison with Random Forest Classifier:

When compared to the Random Forest Classifier, which achieved an accuracy of 80% and an F1 score of 0.80, BERT significantly outperforms the Random Forest model in both accuracy and other metrics. The high precision, recall, and F1 score achieved by BERT highlight its superior performance in handling complex textual data, making it a stronger candidate for deployment in psychiatric classification tasks.

7.3 Analysis of BERT's Improved Performance Over Random Forest

As evident from the performance evaluation, BERT significantly outperforms the Random Forest Classifier. This section analyzes the factors contributing to the substantial performance improvement of BERT over the Random Forest Classifier.

7.3.1 Limitations of Random Forest Classifier in Text Classification

Despite being one of the most effective machine learning algorithms for classification tasks, the Random Forest Classifier exhibits several limitations that hinder its performance in text classification applications.

Feature Representation Challenges

- **Lack of Contextual Understanding:** Random Forest primarily relies on bag-of-words or TF-IDF representations, which do not capture word order or contextual relationships [12]. This limitation is particularly problematic in cases involving:
  - Polysemy (words with multiple meanings)
  - Negations and complex linguistic structures
- **High-Dimensional Sparse Data:** Text data often results in high-dimensional and sparse feature spaces, making it challenging for Random Forest to process efficiently.

Limited Semantic Interpretation

- The Random Forest Classifier struggles to capture long-range dependencies in text.
- It lacks the capability to interpret nuanced language features such as sarcasm and idiomatic expressions.

Preprocessing Requirements

- Extensive preprocessing is required to transform textual data into a structured format suitable for Random Forest.
- This process is time-consuming and may result in the loss of important linguistic information.

Sensitivity to Noise

- The Random Forest model is susceptible to noise in textual data, potentially leading to overfitting on irrelevant features.

Given these limitations, advanced NLP techniques such as BERT are often preferred for complex text classification tasks, particularly when contextual understanding and semantic interpretation are essential.

7.3.2 Advantages of BERT in Text Classification

In German binary text classification, BERT outperforms the Random Forest classifier due to its capability to capture contextual nuances and language-specific patterns via transformer architecture. The following provides a detailed breakdown:

Contextual Understanding

BERT excels in text classification through several key mechanisms:

- **Bidirectional Processing:** BERT processes text bidirectionally, considering both preceding and succeeding words when encoding each word. This enables it to capture the full context of a sentence, unlike traditional models that analyze text in a left-to-right manner. It effectively resolves ambiguities such as:
  - Polysemy: Distinguishing between meanings of words like “bank” (financial vs. river) based on surrounding context.
  - Negations: Detecting phrases like “nicht gut” (not good) in German.
- **Self-attention Mechanism:** BERT uses a self-attention mechanism to weigh the importance of each word based on its context. This allows the model to focus on the most relevant parts of a sentence, enhancing its ability to understand complex relationships and nuances between words.
- **Contextual Word Embeddings:** Unlike traditional word embeddings, BERT generates dynamic contextual word embeddings that change depending on the word’s usage in a sentence. This approach provides a more nuanced understanding of language by representing the same word differently in different contexts.

Language Adaptation

- **Multilingual Pre-training:** BERT's pre-training on multilingual corpora, including German, facilitates transfer learning for domain-specific tasks.
- **Feature Engineering Challenges in Random Forest:** Random Forest relies on manual feature engineering (e.g., n-gram selection), which struggles with:
  - German Morphological Complexity
  - Dialectal Variations (e.g., Bavarian vs. Standard German)

8 Development of AI Assistant

The aim of this project is to develop an AI assistant that helps rescue personnel diagnose psychiatric conditions more effectively. To achieve this, we are developing two AI assistant models: one based on the Random Forest classifier and the other on the fine-tuned BERT model. The goal is to compare the performance of both models on unseen data.

8.1 Assistant with Random Forest Classifier

The first step in the development of the AI assistant involves utilizing the Random Forest classifier for classification. This method is fast and interpretable, making it a strong candidate for deployment in real-world scenarios.

8.1.1 Pipeline

The pipeline for developing the AI assistant using the Random Forest classifier is as follows:

1. Preprocessing User Input

The first step involves processing the user input, which could be in the form of text.

2. Feature Extraction and Feature Vector Using BERT

Once the input is preprocessed, we extract features from the text using BERT. BERT's pre-trained embeddings are ideal for this task, as they capture semantic meaning. This will involve:

- Tokenizing the input using a pre-trained BERT tokenizer.
- Passing the tokenized input to the pre-trained BERT model to extract features.
- Creating a feature vector from the BERT output that captures the important characteristics of the text.

3. Making Prediction Using Random Forest Classifier

Once the feature vector is generated, we use it as input to the Random Forest classifier. The Random Forest model is pre-trained on a dataset of labeled text, with labels indicating whether the patient is psychiatric or non-psychiatric. The classification task is straightforward:

- The model predicts the label based on the feature vector.
- The output is a binary prediction indicating whether the patient's condition is psychiatric or non-psychiatric.

4. Giving Feedback to the User

After the prediction is made, the system will provide feedback to the user. This feedback may include:

- The prediction result (e.g., "The patient is likely psychiatric" or "The patient is likely non-psychiatric").
- **Confidence level:** The probability of the prediction, which can provide additional context to the decision-making process.
- **Suggested actions:** Based on the prediction, the system could provide suggestions for next steps, like contacting medical professionals.

5. Wrapper Function

The assistant function integrates the entire pipeline into a single callable function. It allows rescue personnel to input text, processes the input, makes a prediction, and returns the feedback.

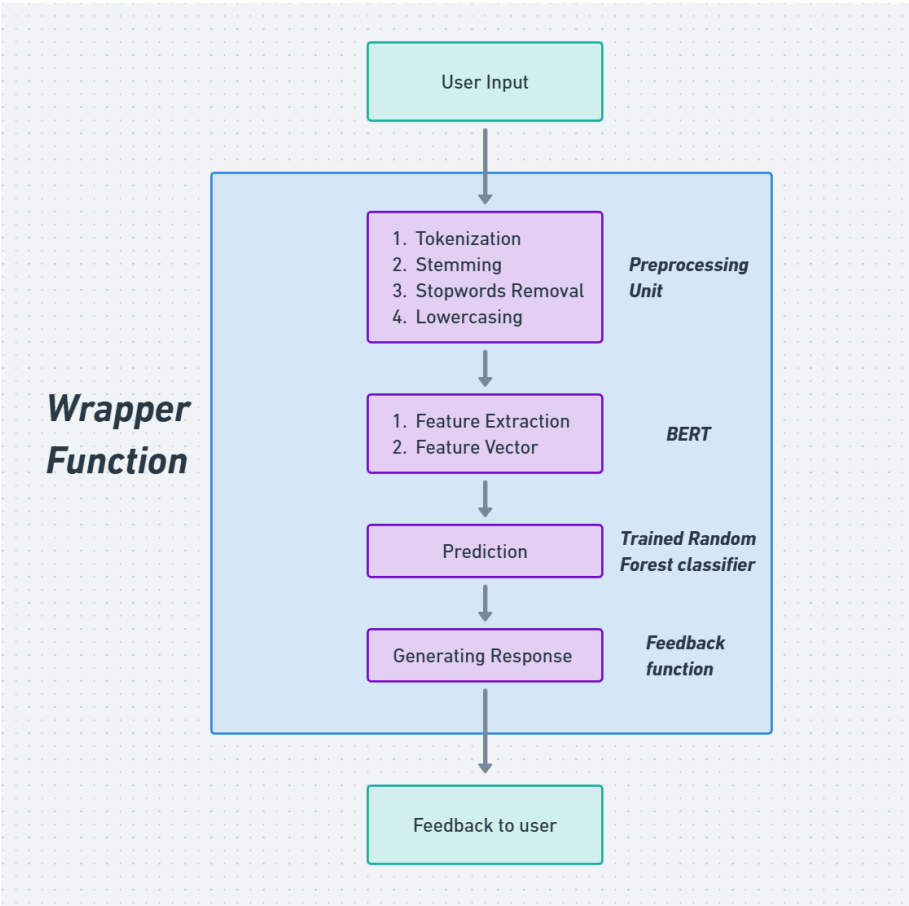

Figure 8.1: AI Assistant with Random Forest Classifier

8.2 Assistant with BERT

In this second version of the AI assistant, we will utilize a fine-tuned BERT model to handle the classification process. BERT, being a transformer-based model, is capable of generating rich word embeddings that capture the semantic meaning of the text. Unlike the Random Forest classifier, which relies on feature engineering, BERT handles the entire text processing and prediction pipeline internally. Below is the outline of the pipeline followed by detailed explanations of each module.

8.2.1 Pipeline

The pipeline for the AI assistant using BERT is as follows:

1. Preprocessing User Input

In this step, we will receive input from the user dynamically. To ensure the input is in a suitable format for BERT, we will preprocess it using the preprocess_text() function we defined earlier.

The function will return preprocessed text that is ready for BERT.

2. Making Prediction Using BERT

In contrast to the Random Forest classifier, BERT does not require explicit feature extraction. Instead, BERT directly generates word embeddings that capture the meaning of each word in the context of the entire sentence. The embeddings are contextual and bidirectional, enabling BERT to understand the relationships between words, even if they appear far apart in the sentence.

Key Features of BERT Embeddings:

1. **Contextual Understanding:** BERT looks at words in both directions (left-to-right and right-to-left), capturing richer semantic meaning.
2. **Subword Tokenization:** BERT uses WordPiece tokenization to break down words into smaller subwords, allowing it to handle rare or out-of-vocabulary words.
3. **Multiple Embedding Layers:** BERT combines token, position, and segment embeddings to understand the full meaning of the input.

To make a prediction:

1. Tokenize the input using BERT's tokenizer.
2. Pass the tokenized input through the pre-trained or fine-tuned BERT model.
3. Use the output embeddings to make a prediction (binary classification: psychiatric or non-psychiatric).

3. Generating Response

Once we have the prediction, we need to generate an appropriate response for the user. The response will vary depending on whether the patient is predicted to be psychiatric or non-psychiatric. This can be done with a simple if-else condition.

4. Wrapper Function

To make the assistant more user-friendly, we will wrap all the individual steps into a single, easy-to-use function. This master function will call the previous modules sequentially and return the final response to the user.

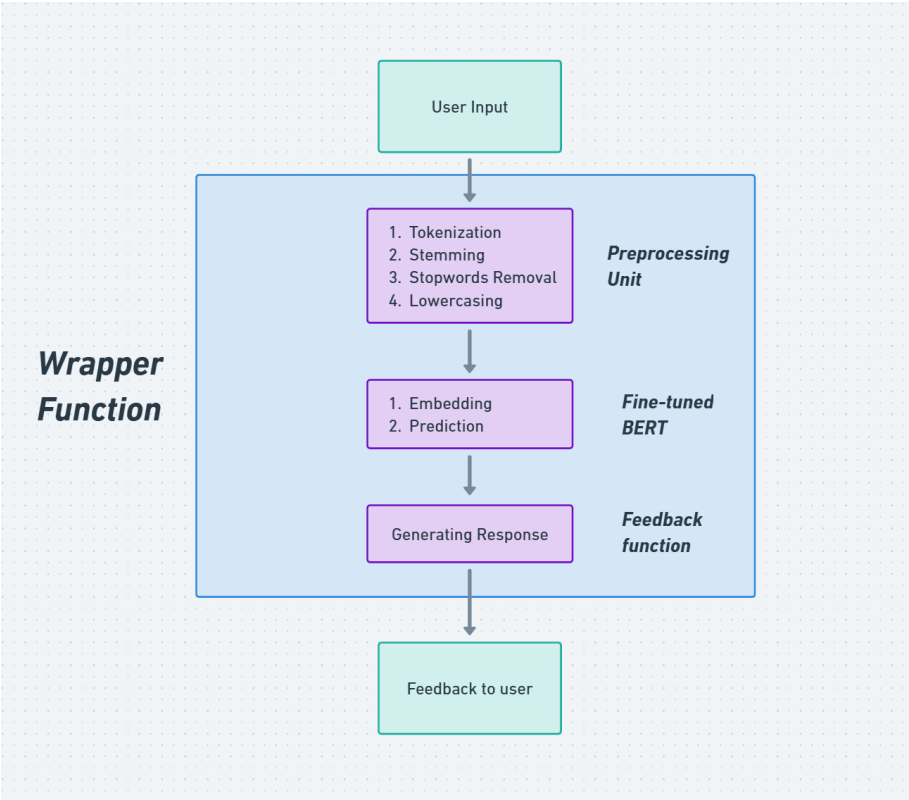

Figure 8.2: AI Assistant with BERT

9 Conclusion

This study demonstrated the transformative potential of artificial intelligence in enhancing emergency medical diagnostics, with a focused application on psychiatric case classification. By leveraging a robust dataset of over 273,000 emergency rescue operations, this project developed an AI assistant combining machine learning (Random Forest) and deep learning (BERT) models to support rescue personnel in rapidly distinguishing psychiatric from non-psychiatric cases.

Key Findings

1. **Model Performance:** The BERT-based model significantly outperformed the Random Forest classifier, achieving an accuracy of 96.4% and an F1-score of 96.4%, compared to Random Forest's 80% accuracy. BERT's bidirectional contextual understanding enabled superior handling of nuanced language and complex symptom descriptions inherent in psychiatric cases.
2. **Feature Engineering:** Custom feature engineering, including synonym mapping, composite features (e.g., combining pain-related terms), and contextual analysis, proved critical for the Random Forest model. However, BERT's ability to process raw text with minimal preprocessing underscored the advantage of transformer-based models in NLP tasks.
3. **Impact on Emergency Care:** The AI assistant reduces cognitive overload for rescue personnel by automating classification, enabling faster interventions. Its natural language interface ensures usability, even for non-technical users, while real-time decision support enhances diagnostic accuracy in high-pressure environments.

Limitations

- The dataset, though extensive, contained inconsistencies due to emergency data collection practices (e.g., typos, missing values).
- The model's focus on psychiatric cases limits its generalizability to other emergency types without retraining.
- Computational demands of BERT may pose challenges for real-time deployment in resource-constrained settings.

Future Directions

1. **Expanded Scope:** Extend the model to classify broader medical emergencies (e.g., cardiovascular, trauma) using multi-label frameworks.
2. **Real-World Integration:** Pilot the assistant in live emergency settings to evaluate workflow compatibility and user feedback.

3. **Interpretability Enhancements:** Develop hybrid models that combine BERT's accuracy with Random Forest's interpretability for clinical transparency.
4. **Multilingual Adaptation:** Train the system on multilingual datasets to support diverse linguistic contexts.

Final Remarks

This project emphasizes AI's potential to transform emergency healthcare by enhancing human decision-making with speed, precision, and scalability. As AI technologies advance, their incorporation into medical workflows holds the promise of addressing crucial gaps in resource-constrained settings, ultimately saving lives through prompt and accurate diagnoses. This assistant, based on BERT, is a clear sign that AI can play a big role in improving global healthcare.

List of Figures

4.1 Diagnosed Cases Count for Different Diseases. 13
4.2 Cases Count Per Class in Final Dataset. 14

5.1 Unigrams 24
5.2 Bigrams 25
5.3 Trigrams 25
5.4 Top 20 Unigrams-Psychiatric Data Frame. 27
5.5 Top 20 Bigrams-Psychiatric Data Frame. 28
5.6 Top 20 Unigrams-Non-psychiatric Data Frame. 28
5.7 Top 20 Bigrams-Non-psychiatric Data Frame. 28
5.8 Word frequency distribution graph 30

6.1 Random Forest Architecture 36
6.2 BERT Architecture 37
6.3 Vectorized Data Frame 40

7.1 Classification Report:Random Forest Classifier 46
7.2 Learning rate:BERT 47
7.3 Training loss:BERT 48
7.4 Accuracy:BERT 48
7.5 Precision:BERT 48
7.6 Recall:BERT 49
7.7 F1-Score:BERT 49

8.1 AI Assistant with Random Forest Classifier 54
8.2 AI Assistant with BERT 56

Bibliography

[1] Abu Shad Ahammed; Aniebiet Micheal Ezekiel; Roman Obermaisser „Time-Efficient Identification Procedure for Neurological Complications of Rescue Patients in an Emergency Scenario Using Hardware-Accelerated Artificial Intelligence Models“. In: *Algorithms* 16 (2023), 5, p. 258.

[2] Abu Shad Ahammed; Roman Obermaisser. „Smart Health Software to Support Rescue Personnel in Emergency Situations“. In: *2024 International Conference on Smart Systems and Technologies (SST)*. 2024, pp. 165–170. DOI: 10.1109/SST61991.2024.10755467.

[3] Sergey K Aityan et al. „Integrated AI Medical Emergency Diagnostics Advising System“. In: *Electronics* 13 (2024), 22, p. 4389.

[4] Christian Arnold et al. „The role of hyperparameters in machine learning models and how to tune them“. In: *Political Science Research and Methods* 12 (2024), 4, pp. 841–848.

[5] Mohammed Awad; Salam Fraihat „Recursive feature elimination with cross-validation with decision tree: Feature selection method for machine learning-based intrusion detection systems“. In: *Journal of Sensor and Actuator Networks* 12 (2023), 5, p. 67.

[6] David W Bates et al. „The potential of artificial intelligence to improve patient safety: a scoping review“. In: *NPJ digital medicine* 4 (2021), 1, p. 54.

[7] Steven Bird. „NLTK: the natural language toolkit“. In: *Proceedings of the COLING/ACL 2006 Interactive Presentation Sessions*. 2006, pp. 69–72.

[8] Francisco Javier Candel; María José Núñez-Orantos „Key issues in the infected patient care in the Emergency Department“. In: *Rev Esp Quimioter* 29 (2016), 6, pp. 318–327.

[9] Gabrielle Chenais; Emmanuel Lagarde; Cédric Gil-Jardiné „Artificial intelligence in emergency medicine: viewpoint of current applications and foreseeable opportunities and challenges“. In: *Journal of Medical Internet Research* 25 (2023). e40031.

[10] Mikhail V Koroteev „BERT: a review of applications in natural language processing and understanding“. In: *arXiv preprint arXiv:2103.11943* (2021).

[11] Ahmet Doğan Kuday et al. „Challenges Faced by Medical Rescue Teams During Disaster Response: A Systematic Review Study“. In: *Disaster Medicine and Public Health Preparedness* 17 (2023). e548.

[12] G. Mahesh et al. „Text Classification Using the Random Forest Algorithm: An Application Study“. In: *International Journal of Computer Engineering and Applications* XVI (2022), X, pp. 103–111.

[13] Shiv Kumar Mudgal et al. „Real-world application, challenges and implication of artificial intelligence in healthcare: an essay“. In: *Pan African Medical Journal* 43 (2022), 1.

[14] Yuris Tri Naili et al. „Regulatory challenges in ai-based diagnostics: Legal implications of ai use in medical diagnostics“. In: *BIO Web of Conferences*. Vol. 152. EDP Sciences. 2025, p. 01034.

[15] Aakash Parmar; Rakesh Katariya; Vatsal Patel. „A review on random forest: An ensemble classifier“. In: *International conference on intelligent data communication technologies and internet of things (ICICI) 2018*. Springer. 2019, pp. 758–763.

[16] Nur Husna Shahimi et al. „Association between mental illness and blood pressure variability: a systematic review“. In: *Biomedical engineering online* 21 (2022), 1, p. 19.

[17] Helene Skjøt-Arkil et al. „Improved diagnostics of infectious diseases in emergency departments: a protocol of a multifaceted multicentre diagnostic study“. In: *BMJ open* 11 (2021), 9, e049606.

[18] Yuli Vasiliev. *Natural language processing with Python and spaCy: A practical introduction*. No Starch Press, 2020.

[19] Li Yujian; Liu Bo „A normalized Levenshtein distance metric“. In: *IEEE transactions on pattern analysis and machine intelligence* 29 (2007), 6, pp. 1091–1095.

[20] Laura Zwaan; Hardeep Singh „The challenges in defining and measuring diagnostic error“. In: *Diagnosis* 2 (2015), 2, pp. 97–103.